Podręcznik: podstawy tworzenia backendu aplikacji na przykładzie Node.js

Strona:	LeIA 2026	Wydrukowane przez użytkownika:	Tymon Kwiatkowski
Kurs:	Zaawansowane aplikacje internetowe (2026Z)	Data:	piątek, 2 października 2026, 12:07
Książka:	Podręcznik: podstawy tworzenia backendu aplikacji na przykładzie Node.js		

Opis

Spis treści

Strona tytułowa

Back-end
- Aplikacja serwerowa
- Protokół HTTP
- Przykład komunikacji klienta z serwerem

Podstawy Node.js
- Instalacja
- Hello, world
- Wypisywanie tekstu
- Tryb watch
- Funkcje
- Obiekty
- Importowanie modułów
- Eksportowanie modułów

Node Package Manager
- Inicjalizacja npm
- Instalacja pakietów
- Skrypty

Asynchroniczność w Node.js
- Callbacks
- Promises

Obsługa błędów w Node.js
- try, catch
- process.on

Operacje na plikach w Node.js
- Wczytywanie plików
- Przetwarzanie ścieżek plików
- Tworzenie folderów
- Tworzenie i modyfikacja plików

Express
- Inicjalizacja Express
- Szkielet aplikacji
- Trasowanie
- Pliki statyczne
- Testowanie aplikacji

Bazy danych
- Podstawy SQL
- Konfiguracja MySQL
- Obsługa MySQL za pomocą Node.js
- Podstawy MongoDB
- Obsługa MongoDB za pomocą Node.js

Inne techniki back-endowe

Strona tytułowa

Niniejszy moduł przedstawia podstawy tworzenia *backendu* na przykładzie Node.js.

Wymagana wiedza bazowa: podstawy tworzenia aplikacji internetowych; znajomość języka JavaScript

Autor: dr inż. Jakub Wagner, Instytut Radioelektroniki i Technik Multimedialnych

Back-end

W kontekście projektowania aplikacji internetowych, ***back-end*** to ta część systemu informatycznego, która odpowiedzialna jest za:

- odbieranie i przetwarzanie *żądań* przychodzących od *klienta*,
- generowanie odpowiedzi na te żądania i wysyłanie ich z powrotem do klienta,

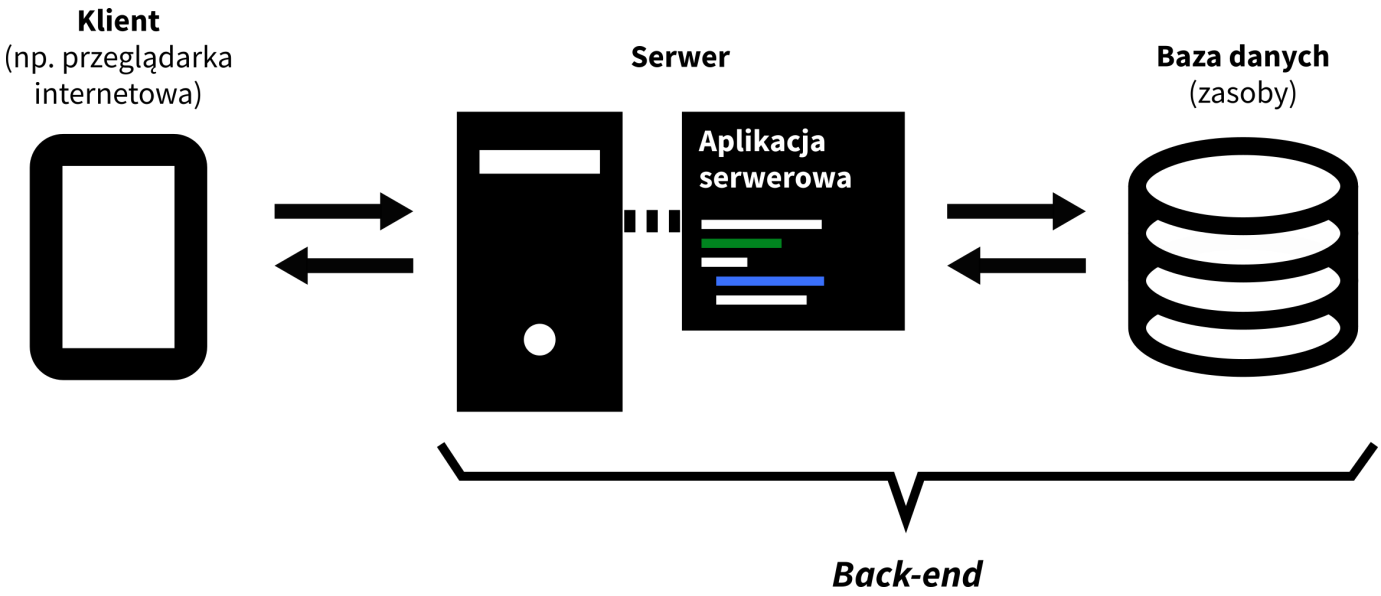
przy czym:

- *żądanie* (ang. ***request***) może mieć na celu pozyskanie lub przetworzenie jakichś zasobów, np. plików,
- *klientem* (ang. ***client***) może być np. przeglądarka internetowa.

Główne składowe *back-endu* to:

- **serwer**, czyli komputer, przetwarzający żądania,
- **aplikacja serwerowa**, czyli oprogramowanie działające na serwerze, które nasłuchuje żądań, pozyskuje zasoby z bazy danych i wysyła odpowiedzi,
- **baza danych**, przechowująca i „organizująca” dane.

Baza danych może znajdować się fizycznie w innym miejscu niż serwer i być z nim połączona przez internet.



Do realizacji *back-endu* typowo wykorzystywane są następujące języki programowania:

- JavaScript (w połączeniu ze środowiskiem Node.js),
- Python,
- Java,
- C#,
- PHP,
- Ruby

i oprócz tego często język SQL – do obsługi baz danych.

Aplikacja serwerowa

Aplikacja serwerowa służy przede wszystkim do generowania odpowiedzi na żądania klientów.

Generacja odpowiedzi dla danego żądania wymaga m.in. określenia tzw. *trasy* (ang. ***route***), stanowiącej zestawienie następujących dwóch elementów:

- określenia tzw. metody żądania (ang. *HTTP method*),
- adresu („ścieżki"), pod który skierowane jest żądanie (ang. *path*, *endpoint*).

Dobór trasy (czyli dobór procedury generacji odpowiedzi dla danego żądania) nosi nazwę *trasowania* (ang. ***routing***).

Odrębnym zadaniem aplikacji serwerowej jest kontrola dostępu do *zasobów* (ang. ***resources***, np. informacji przechowywanych w bazie danych), m.in. poprzez uwierzytelnianie (ang. ***authentication***) użytkowników.

Web API

Możliwości komunikacji między oprogramowaniem po stronie klienta i *back-endem* są określone przez interfejs programistyczny aplikacji serwerowej (ang. ***Web API***). Określa on zbiór żądań, które mogą być obsłużone przez aplikację serwerową oraz definicje odpowiedzi, których można się dla tych żądań spodziewać.

Zasady generowania odpowiedzi na żądania

Generowanie odpowiedzi na żądania przez aplikację serwerową powinno być realizowane zgodnie z następującymi zasadami:

- Odpowiedź powinna być generowana wyłącznie po odebraniu żądania; innymi słowy: przy braku żądań nie powinny być generowane żadne odpowiedzi.
- Na każde żądanie powinna zostać wygenerowana odpowiedź.
- Na pojedyncze żądanie nie powinna zostać wygenerowana więcej niż jedna odpowiedź.

W przypadku otrzymania od klienta żądania o nieprzewidzianej ścieżce, odpowiedź również powinna zostać wygenerowana i wysłana, np. w postaci kodu błędu 404. Brak odpowiedzi może spowodować, że klient będzie na nią czekał w nieskończoność.

Protokół HTTP

HTTP (ang. *Hypertext transfer protocol*) jest standardem (protokołem) komunikacji internetowej między klientami i serwerami. Został on opracowany i jest nadzorowany przez organizację *World Wide Web Consortium* (W3C, www.w3.org). Najnowsza wersja, HTTP/3, została opublikowana w 2022 roku:

Internet Engineering Task Force, Request for Comments 9114 - HTTP/3

Metody HTTP

Standard HTTP charakteryzuje komunikację klient–serwer w postaci „żądanie–odpowiedź” (ang. *request–response*). W szczególności, są w nim wymienione tzw. metody HTTP (ang. ***HTTP methods***), takie jak:

- metoda **GET**, służąca do pozyskiwania zasobów,
- metody **PUT** i **POST**, służące do zapisywania i przetwarzania zasobów,
- metoda **DELETE**, służąca do usuwania zasobów

i inne; pełną listę wraz ze szczegółowymi opisami metod HTTP można znaleźć w specyfikacji standardu.

Metoda HTTP jest podawana jako element żądania, formułowanego przez klienta, i stanowi podstawę do doboru odpowiedniej procedury generacji odpowiedzi przez aplikację serwerową.

Kody odpowiedzi

Standard HTTP określa również liczbowe kody (ang. ***response status codes***), które stanowią część odpowiedzi na żądania, takie jak:

- **200** (*OK*) – żądanie zostało przetworzone z powodzeniem.
- **201** (*Created*) – żądanie poskutkowało utworzeniem jakiegoś zasobu z powodzeniem.
- **301** (*Moved Permanently*) – żądane zasoby zostały przeniesione pod inny adres na stałe.
- **302** (*Found*) – żądane zasoby zostały przeniesione pod inny adres tymczasowo.
- **400** (*Bad Request*) – żądanie nie może być przetworzone z powodzeniem, bo jest błędne (nie jest przewidziane w *Web API*).
- **401** (*Unauthorized*) – żądanie nie może być przetworzone z powodzeniem, bo wysyłający je klient nie jest w odpowiedni sposób autoryzowany.
- **404** (*Not Found*) – żądanie nie może być przetworzone z powodzeniem, bo odnosi się do zasobu, którego nie ma na serwerze (co może mieć miejsce np. w razie podania błędnej nazwy pliku).
- **418** (*I'm a teapot*) – „jestem czajnikiem” (naprawdę).
- **500** (*Internal Server Error*) – żądanie nie zostało przetworzone z powodzeniem, bo w trakcie jego przetwarzania po stronie serwera wystąpił błąd (co może mieć miejsce np. gdy w kodzie aplikacji serwerowej jest błąd programistyczny).

Pełną listę wraz ze szczegółowymi opisami kodów odpowiedzi można znaleźć m.in. w dokumentacji MDN.

Taki kod stanowi element każdej odpowiedzi generowanej przez aplikację serwerową i wysyłanej do klienta.

Przykład komunikacji klienta z serwerem

Przykładowa procedura komunikacji klienta z serwerem w formie *żądanie–odpowiedź* może przebiegać następująco:

- Człowiek wpisuje w przeglądarce internetowej adres www.pw.edu.pl/kontakt.
- Przeglądarka internetowa formułuje żądanie pozyskania treści ze wskazanego wyżej adresu metodą HTTP *GET*.
- Żądanie trafia za pośrednictwem internetu na serwer związany z adresem www.pw.edu.pl [1].
- Aplikacja serwerowa, nasłuchująca aktywnie, otrzymuje wysłane przez przeglądarkę żądanie.
- Aplikacja serwerowa dobiera odpowiednią procedurę („trasę"), związaną ze ścieżką */kontakt* i metodą *GET*.
- W wyniku działania wyżej wymienionej procedury, aplikacja sięga do dostępnych jej zasobów i pobiera z nich dane, związane ze ścieżką */kontakt*: kod HTML, który ma być odczytany i wyświetlony przez przeglądarkę internetową.
- Aplikacja serwerowa generuje odpowiedź, zawierającą m.in. pozyskane dane oraz kod odpowiedzi 200 („*OK*"), po czym odpowiedź tę wysyła do klienta.
- Odpowiedź trafia za pośrednictwem internetu do klienta.
- Klient – przeglądarka internetowa – przetwarza w stosowny sposób odpowiedź i wyświetla otrzymaną z serwera treść.

---

[1] Pojedynczy adres może być obsługiwany przez wiele serwerów. W takim przypadku, serwerem na który trafi żądanie, może być np. ten, który fizycznie znajduje się najbliżej klienta.

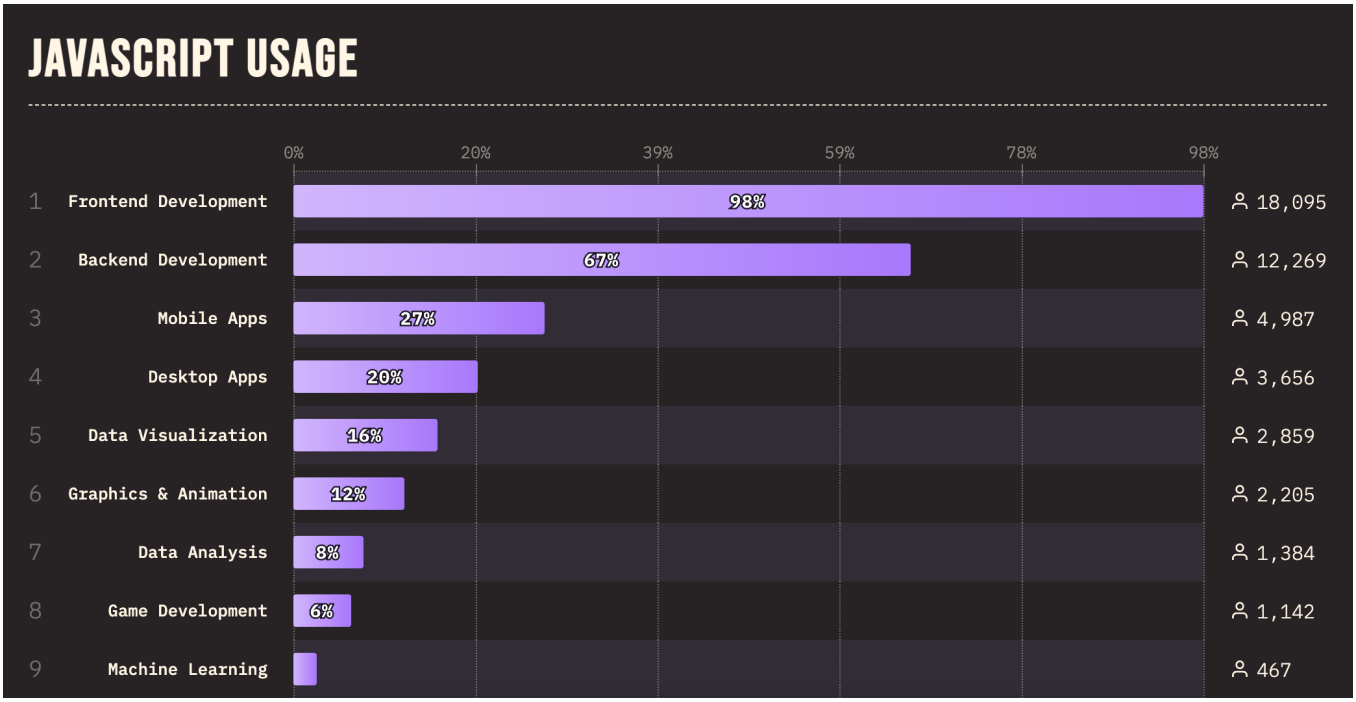
Podstawy Node.js

Node.js jest środowiskiem wykonawczym (ang. *runtime environment*) dla aplikacji w języku JavaScript. Innymi słowy, jest to oprogramowanie, które umożliwia wykonywanie (ang. *execution*) programów w języku JavaScript poza przeglądarkami internetowymi, w szczególności – jako aplikacje serwerowe.

Język JavaScript, opracowany w latach 90. XX w., pierwotnie przeznaczony był do użytku w przeglądarkach internetowych, aby umożliwić wyświetlanie w nich treści dynamicznych. Wciąż jest to najbardziej powszechne zastosowanie tego języka; obecnie jednak używany jest również w innych celach – m.in. do tworzenia aplikacji serwerowych za pomocą środowiska uruchomieniowego Node.js.

Node.js wykorzystuje „silnik" JavaScript V8, opracowany przez Google – ten sam, który jest wykorzystywany w przeglądarce internetowej Chrome.

Statystyki wykorzystania języka JavaScript w 2023 roku:



źródło: State of JavaScript 2023

Najczęściej wymieniane są następujące **zalety środowiska Node.js**:

- umożliwia tworzenie oprogramowania *back-end* w języku JavaScript, który często używany jest do tworzenia oprogramowania *front-end*, dzięki czemu cały projekt aplikacji internetowej może być napisany w jednym języku;
- jest oprogramowaniem otwartym (*open-source*);
- umożliwia łatwą i skuteczną realizację działania asynchronicznego, opartego na zdarzeniach, mimo wykorzystywania pojedynczego procesu; brak konieczności operowania na tzw. wątkach (ang. *threads*) i zarządzania ich tzw. konkurencją zmniejsza ryzyko błędów programistycznych.

Zintegrowane środowiska programistyczne wykorzystywane powszechnie do rozwoju oprogramowania korzystającego z Node.js:

- Visual Studio Code
- WebStorm

Inne zastosowania Node.js

Środowisko Node.js jest wykorzystywane przeważnie do tworzenia *back-endu* stron i aplikacji internetowych. Z drugiej strony, umożliwia ono również m.in.:

- tworzenie klasycznych aplikacji z graficznym interfejsem użytkownika, przeznaczonych na komputery (ang. *desktop apps*); służy do tego platforma programistyczna Electron;
- tworzenie oprogramowania do urządzeń internetu rzeczy.

Instalacja

Instalatory środowiska Node.js dla systemów operacyjnych Windows i MacOS można znaleźć na stronie:

https://nodejs.org/en/download

W systemie operacyjnym Linux środowisko Node.js można zainstalować np. poprzez Snap Store:

https://snapcraft.io/node

Node Version Manager

Oprogramowanie Node Version Manager (nvm), dostępne pod poniższym adresem:

https://github.com/nvm-sh/nvm

pozwala m.in. zainstalować najnowszą wersję środowiska Node.js:

```
> nvm install node
```

instalować wybrane wersje:

```
> nvm install 20.10.0
```

i przełączać się między zainstalowanymi wersjami:

```
> nvm use 20
```

Literatura

- nodejs.org
- Node Version Manager – Intro
- Node Version Manager – Usage
- NVM for Windows

Hello, world

Po zainstalowaniu środowiska Node.js, można je uruchomić w wierszu poleceń za pomocą polecenia `node` i wykonywać w nim polecenia w języku JavaScript.

Wiersz poleceń:

```
> node
```

```
Welcome to Node.js v20.10.0.
Type ".help" for more information.
```

```
> console.log("Hello, world");
```

```
Hello, world
```

Aby opuścić środowisko Node.js, należy dwukrotnie wcisnąć klawisze `CTRL+C`.

Wykorzystanie Node.js polega jednak przede wszystkim na wykonywaniu kodu JavaScript zapisanego w plikach. W wierszu poleceń pliki można wywoływać za pomocą poleceń `node [nazwa_pliku]` .

Jeśli wiersz poleceń jest otwarty w katalogu, w którym znajduje się plik *hello.js* o następującej zawartości:

Plik *hello.js*:

```
console.log("Hello, world");
```

Wówczas zawarty w nim kod można wykonać za pomocą polecenia `node hello.js` :

Wiersz poleceń:

```
> node hello.js
```

```
Hello, world
```

W przypadku plików *.js* można też pominąć ich rozszerzenie:

```
> node hello
```

Wypisywanie tekstu

Instrukcja `console.log(...)` służy do wypisywania tekstu w wierszu poleceń:

```
console.log("Treść komunikatu");
```

```
Treść komunikatu
```

W napisach można uwzględniać wartości zmiennych: napis musi być zawarty wewnątrz „odwrotnych apostrofów” (`` ` ``, ang. *backtick*), a nazwy zmiennych – wewnątrz bloku `${ }`:

```
var message = "Sukces";
console.log(`Treść komunikatu: ${message}`);
```

```
Treść komunikatu: Sukces
```

Wypisywanie komunikatów o błędach

Instrukcja `console.error(...)` służy do wypisywania komunikatów o błędach:

```
console.error("Porażka");
```

```
Porażka
```

Różnica między `console.log` i `console.error` polega na tym, że `console.log` wysyła komunikat do strumienia `stdout`, natomiast `console.error` – do strumienia `stderr`. W przypadku domyślnego uruchomienia środowiska Node.js, oba strumienie jednakowo trafiają do wiersza poleceń.

Tryb watch

Począwszy od **wersji 18.11** środowiska Node.js, możliwe jest uruchamianie aplikacji w tzw. **trybie `watch`**. Jeśli aplikacja jest uruchomiona w tym trybie, to wprowadzenie zmiany w jakimś pliku (np. poprawka kodu JavaScript w pliku *.js*) automatycznie powoduje restart aplikacji, dzięki czemu zmiany są od razu widoczne w jej działaniu. Jest to szczególnie przydatne podczas pracy nad rozwojem aplikacji. Aby skorzystać z tej możliwości, należy wykorzystać polecenie `node --watch [nazwa_pliku]`:

```
> node --watch hello.js
```

Polecenie `node --watch-path=[ścieżka] [nazwa_pliku]` pozwala określić, gdzie Node.js ma wypatrywać zmian – wprowadzanie modyfikacji w plikach nie wymienionych jako `watch-path` nie spowoduje wówczas restartu aplikacji. Można podać więcej niż jedną ścieżkę:

```
> node --watch-path=hello.js --watch-path=my_module.js hello.js
```

Restart aplikacji, wywoływany automatycznie w trybie `watch`, usuwa wcześniejszą zawartość wiersza poleceń. Jeśli chce się ją zachować, można użyć polecenia:

```
> node --watch-preserve-output hello.js
```

Aby zakończyć działanie aplikacji w trybie `watch`, należy wcisnąć klawisze **`CTRL+C`**.

Przed opublikowaniem wersji 18.11 środowiska Node.js, funkcję trybu `watch` realizował popularny **pakiet `nodemon`**. Gdy używa się nowych wersji środowiska Node.js, `nodemon` zwykle nie jest już potrzebny. Wciąż może się jednak niekiedy przydawać, np. gdy korzysta się z języka TypeScript.

Funkcje

W języku JavaScript funkcje można tworzyć za pomocą instrukcji rozpoczynających się od słowa **`function`**:

```
function harmonicMean(x, y) {
  return 2 / (1/x + 1/y);
}
```

Taka instrukcja zawiera:

- nazwę funkcji; w powyższym przykładzie: `harmonicMean`;
- listę argumentów wejściowych funkcji, zawartych w nawiasach okrągłych i oddzielonych przecinkami; w powyższym przykładzie: `(x, y)`;
- instrukcje, które mają być wykonywane po wywołaniu funkcji, zapisane wewnątrz nawiasów klamrowych `{ }`.

Wynik wyrażenia poprzedzonego słowem `return` będzie wartością zwracaną przez funkcję.

Wyrażenia funkcyjne

Funkcje można również tworzyć za pomocą *wyrażeń funkcyjnych* (ang. ***function expressions***):

```
const harmonicMean = function(x, y) {
  return 2 / (1/x + 1/y);
}
```

W powyższym przykładzie, wyrażenie po prawej stronie znaku = jest wyrażeniem funkcyjnym. Wynik tego wyrażenia – funkcja wyznaczająca średnią harmoniczną – jest zapisywany do stałej o nazwie `harmonicMean`.

Funkcja stanowiąca wynik wyrażenia funkcyjnego sama w sobie nie ma nazwy – jest tzw. *funkcją anonimową* (ang. ***anonymous function***). W powyższym przykładzie nazwę ma natomiast zmienna `harmonicMean`, do której zapisywany jest wynik wyrażenia funkcyjnego.

Notacja ze strzałką

Funkcje anonimowe można definiować za pomocą tzw. strzałkowych wyrażeń funkcyjnych (ang. ***arrow function expressions***). W porównaniu ze zwykłymi wyrażeniami funkcyjnymi, wyrażenia strzałkowe:

- są krótsze, m.in. nie zawierają słowa `function`;
- mają jednak pewne ograniczenia (por. MDN Web Docs).

Strzałkowe wyrażenia funkcyjne mają zwykle następującą postać:

```
([lista_argumentów]) => [zwracane_wyrażenie]
```

albo:

```
([lista_argumentów]) => {
  [jedna_lub_więcej_instrukcji]
}
```

Lista argumentów może być pusta, może zawierać jeden argument, albo może zawierać kilka argumentów, oddzielonych przecinkami:

```
() => [zwracane_wyrażenie]

(arg1) => [zwracane_wyrażenie]

(arg1, arg2, arg3) => [zwracane_wyrażenie]
```

W przypadku braku nawiasów klamrowych, wyrażenie znajdujące się po prawej stronie symbolu => jest tym, co w „klasycznej” definicji funkcji byłoby poprzedzone słowem `return`.

Przykładowo, funkcję służącą do obliczania średniej harmonicznej można byłoby zrealizować w następujący sposób:

```
const harmonicMean = (x, y) => 2 / (1/x + 1/y);
```

Istotne jest to, że strzałka jest „podwójna”, tzn. `=>`, a nie `->`.

Przekazywanie funkcji

W języku JavaScript funkcje można przekazywać jako argumenty wejściowe do innych funkcji. Poniższy przykład przedstawia funkcję służącą do wyznaczania przybliżonej wartości pierwszej pochodnej zadanej funkcji matematycznej. Jako argumenty wejściowe przyjmuje ona:

- inną funkcję – tę, którą chcemy zróżniczkować;
- punkt, w którym chcemy wyznaczyć wartość pochodnej.

```
function gradient(f, x) {
  dx = 0.01;
  return (f(x + dx) - f(x - dx)) / (2 * dx);
}
```

Przybliżoną wartość pochodnej funkcji $f(x) = 2x^2 + 3$ w punkcie $x = 1$ można wówczas policzyć i wyświetlić w następujący sposób:

```
function myFunction(x) {
  return 2*x*x + 3;
}

console.log(gradient(myFunction, 1));
```

```
4.0000000000000036
```

Funkcję, której pochodną chcemy policzyć, można byłoby też przekazać do funkcji `gradient` w postaci anonimowej, korzystając z wyrażenia funkcyjnego:

```
var z = gradient( (x) => 2*x*x + 3, 1);
console.log(z);
```

```
4.0000000000000036
```

Literatura

- MDN Web Docs: Javascript functions (Guide)
- MDN Web Docs: Javascript functions (Reference)
- MDN Web Docs: Arrow function expressions

Obiekty

Obiekty są w języku JavaScript podstawowym sposobem przechowywania danych w postaci par „klucz-wartość”. Obiekt jest zbiorem ***własności*** (ang. ***properties***). Własność obiektu może zawierać konkretną wartość (np. liczbową albo tekstową); może też być funkcją. Funkcje stanowiące własności obiektu nazywane są ***metodami*** (ang. ***methods***) tego obiektu. Obiekty można definiować za pomocą nawiasów klamrowych `{ }`:

```
var robot = {
  name: "R2-D2",
  dateCreated: new Date(),
  sayHello: () => console.log("beep"),
  addNumbers: (x, y) => x + y,
};
```

W definicji obiektu definicje kolejnych własności muszą być oddzielone przecinkami. Po ostatniej własności może (ale nie musi) się również znajdować przecinek.

Do własności obiektu – w tym również metod – można się odwoływać za pomocą wyrażeń postaci `[nazwa_obiektu].[nazwa_własności]`:

```
console.log(robot.name);
```

```
R2-D2
```

```
console.log(robot.dateCreated);
```

```
2023-12-19T11:09:14.325Z
```

```
robot.sayHello();
```

```
beep
```

```
var sum = robot.addNumbers(3, 4);
console.log(sum);
```

```
7
```

W ten sposób można również modyfikować własności obiektu:

```
robot.dateCreated = new Date(1977);
```

...a także dodawać do obiektu nowe własności:

```
robot.homeworld = "Naboo";
```

Za pomocą polecenia `console.log` można wypisać wszystkie własności obiektu:

```
console.log(robot);
```

```
{
  name: 'R2-D2',
  dateCreated: 1977-01-01T00:00:01.970Z,
  sayHello: [Function: sayHello],
  addNumbers: [Function: addNumbers],
  homeworld: 'Naboo'
}
```

Notacja strzałkowa *vs.* `function`

Metody obiektu można definiować zarówno za pomocą notacji strzałkowej, jak i słowa `function`. Poniższe trzy instrukcje są równoważne:

```
var robot = {
  addNumbers: (x, y) => x + y,
};

var robot = {
  addNumbers: function(x, y) {
    return x + y;
  },
};

var robot = {
  addNumbers(x, y) {
    return x + y;
  },
};
```

Między notacją strzałkową i „klasyczną" jest jednak istotna różnica: metody tworzone za pomocą notacji strzałkowej nie mają dostępu do innych własności obiektu, w którym się znajdują. Rozważmy przykład obiektu reprezentującego kwadrat o zadanych współrzędnych środka i długości boku. Obiekt ten zawiera metodę służącą do sprawdzania, czy zadany punkt znajduje się wewnątrz tego kwadratu:

```
var square = {

  center: {
    x: 1.0,
    y: 0.0,
  },

  side: 1.0,

  contains: function(x, y) {
    var dx = Math.abs(x - this.center.x);
    var dy = Math.abs(y - this.center.y);
    return ((dx <= this.side / 2) && (dy <= this.side / 2));
  }
}
```

W powyższym fragmencie kodu, w metodzie `contains` wykorzystane są własności kwadratu: współrzędne środka i długość boku. Dostęp do tych własności jest możliwy za pomocą słowa `this`: oznacza ono obiekt, w którym jest zdefiniowana dana metoda.

```
console.log(square.contains(0.5, 0.0));
```

```
true
```

Zmiana współrzędnych środka będzie odzwierciedlona w wynikach metody `contains`, ponieważ przy każdym wywołaniu metoda ta sięga do bieżących wartości własności kwadratu:

```
square.center.y = 3.0;
console.log(square.contains(0.5, 0.0));
```

```
false
```

Realizacja powyższego programu nie byłaby możliwa za pomocą notacji strzałkowej, ponieważ wewnątrz strzałkowych wyrażeń funkcyjnych nie można używać słowa `this`:

```
var square = {

  center: {
    x: 1.0,
```

```
    y: 0.0,
  },

  side: 1.0,

  // Błąd: "this" nie działa w funkcjach strzałkowych
  contains: (x, y) => (Math.abs(x - this.center.x) <= this.side / 2) &&
    (Math.abs(y - this.center.y) <= this.side / 2),
}

console.log(square.contains(0.5, 0.0));
```

```
TypeError: Cannot read property 'x' of undefined
```

Literatura

- MDN Web Docs: Javascript object basics
- MDN Web Docs: Working with objects

Importowanie modułów

Podstawowe możliwości środowiska Node.js można rozszerzyć za pomocą tzw. dodatkowych modułów (ang. *modules*). Można wyróżnić trzy rodzaje modułów:

- moduły „bazowe”, tzn. dostępne w ramach podstawowej instalacji Node.js,
- moduły „zewnętrzne”, instalowane m.in. za pomocą Node Package Manager (*npm*),
- moduły tworzone własnoręcznie.

Moduły *CommonJS*

W Node.js powszechnie używane są tzw. **moduły *CommonJS***. Ich importowanie realizowane jest za pomocą instrukcji **`require`**, np.:

```
const os = require('os');  // typowe dla Node.js
```

Powyższe polecenie skutkuje zaimportowaniem modułu `os` (od ang. akronimu *operating system*, system operacyjny).

Ten rodzaj modułów różni się od modułów stosowanych powszechnie w JavaScript poza środowiskiem Node.js, czyli tzw. **modułów *ECMAScript* (ES)**. Node.js **umożliwia również obsługę modułów ES**. Importuje się je za pomocą instrukcji **`import`**, np.:

```
import * from module_name;  // powszechne w JavaScript oprócz Node.js
```

Importowanie różnych rodzajów modułów

Moduły „bazowe” (dostępne od momentu instalacji środowiska Node.js), a także moduły instalowane za pomocą *npm*, można importować podając samą ich nazwę, np.:

```
const events = require('events');
```

Z kolei aby zaimportować moduł stworzony własnoręcznie i zapisany w pliku *.js*, należy podać względną ścieżkę pliku, np.:

```
const my_module = require('./my_module.js')
// zakładając, że plik 'my_module.js' znajduje się w tym samym
// folderze, co ten plik, w którym następuje import
```

Używanie zaimportowanego modułu

Po zaimportowaniu modułu za pomocą polecenia:

```
const [nazwa_modułu] = require('[nazwa_modułu]');
```

można korzystać z zawartych w nim funkcji za pomocą poleceń typu:

```
[nazwa_modułu].[nazwa_funkcji](...)
```

Przykładowo, w module `os` znajduje się funkcja `version`, służąca do pozyskiwania informacji o wersji używanego systemu operacyjnego. Można jej użyć w następujący sposób:

```
const os = require('os');
console.log(os.version());
```

```
Windows 10 Pro
```

Importowanie wybranych elementów modułu

Przeważnie w importowanym module znajdują się różne funkcje. Jeśli potrzebne są tylko niektóre z nich, można zaimportować wybrane funkcje za pomocą poleceń typu:

```
const { [nazwa_funkcji_1], [nazwa_funkcji_2], ... } = require('[nazwa_modułu]');
```

Wówczas z zaimportowanych funkcji można korzystać bez odwoływania się do nazwy modułu. Na przykład:

```
const { version } = require('os');
console.log(version());  // samo "version", nie "os.version"
```

Literatura

- Dokumentacja Node.js: *CommonJS modules*
- Dokumentacja Node.js: *ECMAScript modules*

Eksportowanie modułów

Tworzenie modułów własnoręcznie może służyć m.in. do porządkowania kodu programu: definicje różnych funkcji można umieścić w jednym pliku, a wywoływać je w innym. Przykładowo, można stworzyć moduł, zawierający funkcje realizujące różne działania na zmiennych tekstowych:

Plik *utils.js*

```
// odwracanie kolejności znaków
function backwards(text) {
  return text.split('').reverse().join('');
}

// usuwanie spacji
function removeSpaces(text) {
  return text.split(" ").join("");
}

// sprawdzanie, czy tekst jest palindromem
function isPalindrome(text) {
  var textWithoutSpaces = removeSpaces(text.toLowerCase());
  return textWithoutSpaces === backwards(textWithoutSpaces);
}
```

Aby określić, które funkcje mają być dostępne po zaimportowaniu modułu, należy tę informację zamieścić w **obiekcie `module`**. Obiekt `module` zawiera **własność `exports`**, która również jest obiektem. Funkcje, które mają być wyeksportowane, należy uczynić metodami obiektu `module.exports`:

```
module.exports.backwards = backwards;
module.exports.isPalindrome = isPalindrome;
```

Alternatywnie, wszystkie funkcje przeznaczone do wyeksportowania można zapisać w obiekcie `module.exports` za pomocą pojedynczego polecenia (umieszczanego zwykle na końcu pliku):

```
module.exports = { backwards, isPalindrome };
```

W odrębnym pliku można zaimportować stworzony moduł za pomocą polecenia `require`:

```
const utils = require('./utils.js');
// zakładając, że plik "utils.js" jest w tym samym katalogu, co ten plik
```

i korzystać z dostępnych w nim funkcji:

```
var palindrome = "Jeż largo gra lżej";
console.log(utils.backwards(palindrome))
console.log(utils.isPalindrome(palindrome))
```

```
jeżl arg ogral żeJ
true
```

Cały kod opisanego powyżej przykładowego programu, zawarty w dwóch plikach: *utils.js* i *test.js*, jest przedstawiony poniżej:

Plik *utils.js*

```
// odwracanie kolejności znaków
function backwards(text) {
  return text.split('').reverse().join('');
}
```

```
// usuwanie spacji
function removeSpaces(text) {
  return text.split(" ").join("");
}

// sprawdzanie, czy tekst jest palindromem
function isPalindrome(text) {
  var textWithoutSpaces = removeSpaces(text.toLowerCase());
  return textWithoutSpaces === backwards(textWithoutSpaces);
}

module.exports = { backwards, isPalindrome };
```

Plik *test.js*

```
const utils = require('./utils.js');

var notPalindrome = "To nie jest palindrom";
var palindrome = "Jeż largo gra lżej";

console.log(utils.backwards(notPalindrome))
console.log(utils.isPalindrome(notPalindrome))

console.log(utils.backwards(palindrome))
console.log(utils.isPalindrome(palindrome))
```

Wiersz poleceń

```
> node test
```

```
mordnilap tsej ein oT
false
jeżl arg ogral żeJ
true
```

Własności obiektu `module.exports` nie muszą być metodami; można eksportować również stałe:

```
const ver = '1.7';
module.exports.ver = ver;
```

Literatura

- Dokumentacja Node.js: CommonJS modules
- Dokumentacja Node.js: *The module object*

Node Package Manager

Node Package Manager (**npm**) to popularny system obsługi pakietów dla środowiska Node.js, składający się z trzech elementów:

- rejestru pakietów przeznaczonych dla Node.js,
- strony internetowej, służącej m.in. do wyszukiwania i udostępniania takich pakietów,
- oprogramowania przeznaczonego do użytku w wierszu poleceń (ang. *Command Line Interface*), służącego do pobierania i obsługi takich pakietów.

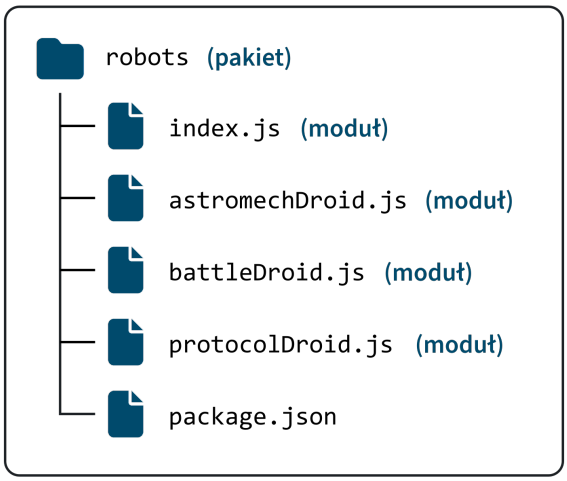
Oprogramowanie npm zwykle instalowane jest razem ze środowiskiem Node.js, na przykład przy użyciu Node Version Manager (nvm). Instrukcje dotyczące instalacji npm można znaleźć w dokumentacji.

Dokumentacja npm:
https://docs.npmjs.com/

Pakiety *vs.* moduły

W Node.js **moduł** (ang. ***module***) oznacza pojedynczy plik, nadający się do zaimportowania (w szczególności: plik *.js*).

Każdy **pakiet** (ang. ***package***) systemu npm zawiera jeden lub więcej powiązanych ze sobą modułów. Może mieć postać folderu, zawierającego pliki *.js*; może też zawierać pojedynczy plik *.js*. Pakiet musi zawierać m.in. plik *package.json* (opisany w dalszej części tego rozdziału).



Dokumentacje Node.js i npm wydają się niespójne w kwestii rozumienia słowa „moduł". W dokumentacji Node.js można znaleźć następujące stwierdzenie:

> *In Node.js, each file is treated as a separate module.*

Natomiast w dokumentacji npm – następujące:

> *A module is any file or directory in the `node_modules` directory that can be loaded by the Node.js `require()` function.*

Inicjalizacja npm

Aby zainicjalizować obsługę pakietów dla danego projektu, należy otworzyć wiersz poleceń w folderze odpowiadającym temu projektowi, a następnie wpisać polecenie:

```
> npm init
```

Polecenie to skutkuje wykonaniem pewnych czynności, dzięki którym bieżący projekt przybierze postać nowego pakietu (w szczególności – zostanie stworzony plik *package.json*, opisujący ten nowy pakiet). W rezultacie, po pierwsze, będzie można importować i wykorzystywać inne pakiety za pomocą npm; po drugie, w przyszłości będzie można ten nowo utworzony pakiet udostępnić za pomocą npm.

Po wywołaniu polecenia `npm init`, oprogramowanie npm zada szereg pytań dotyczących nowo powstałego pakietu, w szczególności:

- o nazwę – domyślnie: nazwa bieżącego folderu;
- o wersję – domyślnie: 1.0.0;
- o plik *entry point*, czyli „główny" moduł, który zostanie zaimportowany, gdy ten pakiet zostanie w przyszłości wczytany za pomocą polecenia `require` – domyślnie: *index.js*, jeśli taki plik istnieje;
- o licencję – domyślnie: licencja Internet Systems Consortium (ISC).

a także o inne informacje, których podanie nie jest konieczne, jak m.in.:

- opis,
- repozytorium git,
- słowa kluczowe,
- autor.

Aby uniknąć odpowiadania na wyżej wymienione pytania i przyjąć wszystkie domyślne wartości, można skorzystać z opcji `--yes`:

```
> npm init --yes
```

albo krócej:

```
> npm init -y
```

Plik package.json

Plik *package.json* zawiera różne informacje, potrzebne do działania npm, w szczególności: informacje o bieżącym pakiecie oraz listę wszystkich pakietów, które są w nim wykorzystywane. W wielu sytuacjach plik *package.json* jest zarówno odczytywany, jak i modyfikowany automatycznie przez npm. Przykładowa zawartość pliku *package.json*:

```
{
  "name": "my_package",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "start": "node ./bin/www"
  },
  "dependencies": {
    "cookie-parser": "~1.4.4",
    "debug": "~2.6.9",
    "express": "~4.16.1",
    "http-errors": "~1.6.3",
    "morgan": "~1.9.1",
    "pug": "2.0.0-beta11"
  }
}
```

Instalacja pakietów

Aby zainstalować pakiet za pomocą oprogramowania npm, tak, aby mógł być wykorzystany w bieżącym projekcie, należy w wierszu poleceń wywołać polecenie `npm install [nazwa_pakietu]`, np.:

```
> npm install chalk
```

albo polecenie `npm install [nazwa_pakietu]@[numer_wersji]`, aby zainstalować wybraną wersję pakietu:

```
> npm install chalk@4
```

Pakiet `chalk` służy do wypisywania kolorowego tekstu w wierszu poleceń. Jego najnowsza wersja ma postać modułu ES. Wersja 4 jest ostatnią, która ma postać modułu CommonJS.

Instalacja pakietu za pomocą npm ma następujące skutki:

- zależność (ang. *dependency*) od tego pakietu zostaje zarejestrowana na liście `dependencies` w pliku *package.json*;
- pliki związane z zainstalowanym pakietem zostają umieszczone w folderze *node_modules* (który zostaje stworzony, jeśli wcześniej nie istniał);
- zostaje zmodyfikowany (lub stworzony) plik *package-lock.json*, do którego lepiej w ogóle nie zaglądać – jest on w pełni obsługiwany automatycznie przez npm.

Polecenie `npm install` (bez żadnej konkretnej nazwy pakietu) sprawia, że npm „zagląda" do pliku *package.json* i na podstawie jego treści instaluje wszystkie wymagane pakiety.

```
> npm install
```

```
added 5 packages, removed 1 package, changed 1 package, and audited 7 packages in 1s
```

Pakiety + git

Jeśli wykorzystywany jest system kontroli wersji git, **w zdalnym repozytorium:**

- **powinien** się znaleźć plik ***package.json***;
- **nie powinny** się znaleźć pliki pakietów zapisane w folderze ***node_modules***.

Idea jest następująca: przy klonowaniu lub aktualizowaniu repozytorium, pobierany jest plik *package.json*, zawierający informację o wszystkich wymaganych pakietach. Po tym należy wywołać polecenie `npm install`, aby te wymagane pakiety osobno pobrać i zainstalować. W ten sposób informacja o wymaganych pakietach jest zachowana w zdalnym repozytorium, ale same pliki pakietów nie zajmują w nim niepotrzebnie miejsca. Z tego powodu, folder *node_modules* należy uwzględnić w pliku *.gitignore*.

Plik *.gitignore*:

```
...

node_modules/

...
```

Wykorzystywanie zainstalowanych pakietów

Aby wykorzystać zainstalowany pakiet, należy go w pliku *.js* zaimportować:

```
const chalk = require('chalk');
console.log(chalk.green('Hello, world'));
```

```
Hello, world
```

Odinstalowywanie pakietów

Do odinstalowania wybranego pakietu służy polecenie `npm uninstall [nazwa_pakietu]`, w skrócie `un` lub `rm`:

```
> npm rm chalk
```

Odinstalowany pakiet zostaje usunięty z pliku *package.json*, ale instrukcje `require` służące do jego importu w plikach *.js* nie zostają automatycznie usunięte.

Literatura

- Dokumentacja npm: *Specifying dependencies and devDependencies in a package.json file*
- Stack Overflow: *What's the difference between dependencies, devDependencies, and peerDependencies in NPM package.json file?*

Skrypty

Skrytpty systemu npm służą do uruchamiania aplikacji – zarówno podczas pracy nad nią, jak i „w produkcji”, tzn. w sytuacji, w której aplikacja jest dostępna dla docelowych użytkowników, np. działa na serwerze.

Skrypty umieszcza się w pliku *package.json* na liście `scripts`.

Plik *package.json*

```
{
  "name": "my_package",
  ...
  "scripts": {
    "start": "node app.js",
    "dev": "node --watch app.js"
  },
  ...
}
```

Aby wywołać skrypt, należy użyć polecenia `npm run [nazwa_skryptu]` w wierszu poleceń:

```
> npm run dev
```

Zgodnie z przykładowym plikiem *package.json*, przedstawionym wyżej, powyższe polecenie poskutkuje wywołaniem następującego polecenia:

```
> node --watch app.js
```

Zwykle skrypt służący do „docelowego” uruchomienia aplikacji (tzn. w takiej formie, w jakiej ma ona trafić do użytkowników, która może się trochę inna, niż podczas pracy nad aplikacją) nazywa się `start`. W npm istnieje specjalne polecenie, służące do wywoływania tego skryptu:

```
> npm start
```

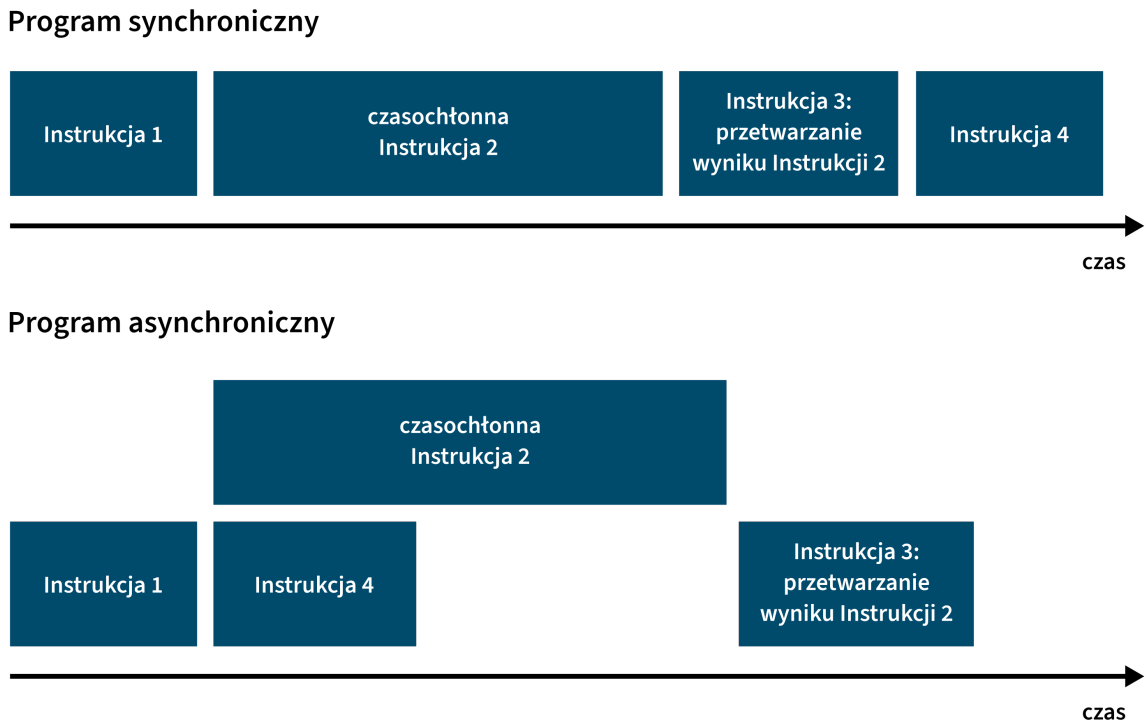
równoważne z poleceniem:

```
> npm run start
```

Asynchroniczność w Node.js

Asynchroniczność oznacza sytuację, w której program „robi kilka rzeczy równocześnie”.

W programie **synchronicznym** kolejność wykonywania instrukcji jest znana i ustalona – jest to ta sama kolejność, w jakiej są one zapisane w kodzie programu. Z kolei w programie **asynchronicznym** czas, w którym instrukcje będą wykonywane, może być zupełnie inny, niż wynikałby z kolejności, w jakiej instrukcje widnieją w kodzie. Programowanie asynchroniczne jest szczególnie użyteczne, gdy jakaś instrukcja jest czasochłonna, np. wymaga skomplikowanych obliczeń albo komunikacji przez internet. Umieszczenie takiej instrukcji w programie synchronicznym mogłoby poskutkować zablokowaniem jego wykonania na dłuższą chwilę, natomiast programowanie asynchroniczne umożliwia to, że po wywołaniu czasochłonnej instrukcji program przechodzi do innych czynności, po czym wraca do przetwarzania wyników tamtej czasochłonnej instrukcji dopiero wtedy, gdy zakończy ona swoje działanie. Korzyścią z takiego obrotu spraw jest m.in. uniknięcie zablokowania działania programu – np. obsługi interfejsu użytkownika.



W JavaScript – zarówno „przeglądarkowym”, jak i w Node.js – są dwie podstawowe techniki programowania asynchronicznego:

- za pomocą tzw. funkcji ***callback***,
- za pomocą tzw. *obietnic* (ang. ***promises***).

Używanie *promises* jest obecnie dość powszechnie uznawane za najlepsze podejście, niemniej programowanie asynchroniczne za pomocą funkcji *callback* wciąż jest często stosowane.

W środowisku Node.js asynchroniczność jest realizowana *jednowątkowo*, za pomocą tzw. pętli zdarzeń (ang. ***event loop***). Jest to powszechnie uznawane za jego zaletę, bo programowanie za pomocą wątków jest trudniejsze niż za pomocą zdarzeń.

Literatura

- MDN Web Docs: Introducing asynchronous JavaScript
- Marijn Haverbeke, *Eloquent Javascript*, Chapter 11: Asynchronous Programming

Callbacks

Asynchroniczna funkcja jako jeden z argumentów wejściowych może przyjmować tzw. funkcję *callback*, która ma zostać wywołana po zakończeniu jej działania. Przykładowo, funkcję `readFile` z modułu `fs` można wywołać w następujący sposób:

```
fs.readFile([ścieżka_pliku], [callback]);
```

Funkcja ta wczytuje plik z podanej ścieżki, a następnie wywołuje funkcję `[callback]`. Nie wiadomo, kiedy ta funkcja zostanie wywołana; wiadomo tylko, że wydarzy się to po wczytaniu pliku. Kod programu, który znajduje się pod wywołaniem `readFile`, może (choć nie musi) zostać wykonany zanim zostanie wywołana funkcja `[callback]`. Na przykład:

```
const fs = require('fs');

fs.readFile("my_file.txt", (err, data) => {
  if (err) throw err;
  console.log("Plik został wczytany");
});

console.log("Funkcja readFile została wywołana")
```

```
Funkcja readFile została wywołana
Plik został wczytany
```

Piramidy zguby

Problem z używaniem funkcji *callback* pojawia się, gdy kilka operacji asynchronicznych trzeba wywołać po kolei – np. sięgać kilka razy do bazy danych, za każdym razem korzystając z wyników poprzednich zapytań. Użycie funkcji *callback* wymaga wówczas zagnieżdżania wielu funkcji, co może pogorszyć czytelność kodu:

```
function zróbCośZłożonego(dane) {
  operacja1(dane, (wynik1) => {
    operacja2(wynik1, (wynik2) => {
      operacja3(wynik2, (wynik3) => {
        operacja4(wynik3);
      });
    });
  });
}
```

Kod programu, zawierający liczne zagnieżdżone funkcje *callback* – jak powyżej – jest trudny w utrzymaniu. Jeśli wystąpi w nim błąd, to trudno jest ten błąd zidentyfikować. Między innymi z tego powodu styl programowania, przedstawiony w powyższym przykładzie, często opatrywany jest nazwami mającymi zniechęcić do jego naśladowania, jak np. *callback hell* albo *pyramid of doom*. Używanie *promises* (opisane w następnym podrozdziale) pozwala na realizację asynchroniczności wolną od tego problemu.

Literatura

- MDN Web Docs: Introducing asynchronous JavaScript
- Marijn Haverbeke, *Eloquent Javascript*, Chapter 11: Asynchronous Programming

Promises

Obiekty *promise* (pol. *obietnica*) reprezentują operacje asynchroniczne, które mogą w pewnym momencie zakończyć się i zwrócić jakąś wartość. *Promise* „obiecuje" (albo „zapowiada") wartość, o której nie wiadomo, w którym momencie stanie się dostępna. Przetwarzanie wartości „obiecanych" przez *promise* odbywa się asynchronicznie, wtedy, kiedy wartości te stają się dostępne. Kod programu opartego na *promises* może przypominać prosty kod synchroniczny i to stanowi jedną z głównych zalet tej techniki programowania asynchronicznego.

Przykładowo, pracę z *promises* umożliwia **API `promises`** modułu `fs`, dostępne w Node.js jako alternatywa dla „tradycyjnych" funkcji modułu `fs` (w których asynchroniczność jest realizowana za pomocą funkcji *callback*). Funkcja `readFile`, zawarta w API `promises`, zwraca obiekt *promise*:

```
const { readFile } = require('fs').promises;

// Utworzenie obiektu promise, reprezentującego "obietnicę" wczytania pliku
const promise = readFile("my_file.txt", "utf8");
```

Aby określić, co ma się dziać po „spełnieniu obietnicy" – czyli wczytaniu pliku – można wykorzystać **metodę `then`**, która jako argument przyjmuje funkcję:

```
promise.then((data) => {
  // Poniższa instrukcja zostanie wykonana po spełnieniu obietnicy,
  // czyli po wczytaniu pliku
  console.log(`Zawartość pliku:\n${data}`);
});
```

Aby określić, co ma się dziać, jeśli obietnicy nie uda się spełnić (czyli zostanie *odrzucona* – np. w przypadku, gdy podana zostanie błędna nazwa pliku), można wykorzystać **metodę `catch`**:

```
promise.then((data) => {
  console.log(`Zawartość pliku:\n${data}`);
}).catch((error) => {
  // Poniższa instrukcja zostanie wykonana, jeśli obietnica
  // zostanie odrzucona (rejected), tzn. nie uda się wczytać pliku
  console.error(`Nie udało się wczytać pliku, bo:\n${error}`);
});
```

`async`, `await`

Słowa `async` i `await` umożliwiają wygodny sposób pracy z obiektami *promise*, nie wymagający ani tworzenia ich wprost, ani jawnego wywoływania metody `then`.

Poprzedzenie definicji funkcji słowem `async` sprawia, że funkcja ta będzie zwracała obiekt *promise* (a co za tym idzie, jej działanie będzie realizowane asynchronicznie):

```
// Funkcja main będzie zwracała promise
async function main() { ... }
```

Wewnątrz funkcji opatrzonej słowem `async` można wykorzystywać słowo `await`, aby sprawić, że program będzie „oczekiwał" na spełnienie jakiejś *promise*, zanim wykonane zostaną jego kolejne instrukcje:

```
const { readFile } = require('fs').promises;

async function main() {

  const data = await readFile("my_file.txt", "utf8");

  // Poniższa instrukcja zostanie wywołana dopiero
  // po spełnieniu obietnicy przez readFile
```

```
  console.log(`Zawartość pliku:\n${data}`);
}

main()
```

Użycie słowa async nieodłącznie oznacza użycie obiektu *promise*. Program przedstawiony w powyższym przykładzie jest – z grubsza – równoważny programowi przedstawionemu wcześniej, korzystającemu z metody `then`.

Za pomocą słów `async`, `await` można w czytelny sposób zrealizować serię operacji asynchronicznych, z których każda korzysta z poprzednich wyników:

```
async function zrobCośZłożonego(dane) {
  wynik1 = await operacja1(dane)
  wynik2 = await operacja2(wynik1)
  wynik3 = await operacja3(wynik2)
  operacja4(wynik3);
}
```

Jedną z głównych zalet *promises* realizowanych za pomocą słów `async` i `await` jest możliwość pisania kodu asynchronicznego, który przypomina kod synchroniczny (a więc jest czytelny), jak w przykładzie powyżej.

Stany *promises*

Obiekt *promise* znajduje się zawsze w jednym z trzech stanów:

- nierozstrzygnięty (ang. ***pending***),
- spełniony (ang. ***fulfilled***),
- odrzucony (ang. ***rejected***).

Stan *pending* jest początkowym stanem każdej *promise*. Oznacza, że „prace trwają” (albo nawet jeszcze się nie zaczęły) i nie wiadomo, jaki będzie ostateczny stan i wartość tej *promise*, ani kiedy ten stan i wartość zostaną określone.

Stan *fulfilled* oznacza, że zapowiadana operacja została zakończona i dostępna jest „obiecana” wartość (***fulfillment value***).

Stan *rejected* oznacza, że zapowiadana operacja nie powiodła się, ale dostępna jest informacja o przyczynie odrzucenia (***rejection reason*** – coś w rodzaju wyjątku, który można „przechwycić”).

W kontekście *promises* pojawia się często określenie ***resolve***. W szczególności, istnieje użyteczna funkcja `Promise.resolve`. Wbrew pozorom, określenie *resolve* niekoniecznie oznacza „spełnienie” *promise* w sensie wprowadzenia jej w stan *fulfilled*. *Promise* może być *resolved* jakiś czas przed „rozstrzygnięciem” jej stanu, może być równocześnie *resolved* i *pending*, a także równocześnie *resolved* i *rejected*.

To, że *promise* jest *resolved*, oznacza że:

- albo dana *promise* jest „rozstrzygnięta”, czyli jest *fulfilled* z określoną wartością lub *rejected* z określonym powodem;
- albo docelowy stan i wartość danej *promise* zależy wyłącznie od docelowego (nieznanego jeszcze) stanu i wartości jakiejś innej, „nierozstrzygniętej” *promise*.

Literatura

- MDN Web Docs: Introducing asynchronous JavaScript
- MDN Web Docs: Using Promises
- MDN Web Docs: How to use promises
- Marijn Haverbeke, *Eloquent Javascript*, Chapter 11: Asynchronous Programming
- JavaScript: The New Toys, *Let's talk about how to talk about promises*

Obsługa błędów w Node.js

Obsługę błędów w Node.js typowo realizuje się za pomocą następujących technik:

- **instrukcji `try`, `catch`** – w przypadku kodu synchronicznego albo kodu zawierającego instrukcje `async`, `await` (a więc wykorzystującego *promises*);
- **obiektu `process`** – w przypadku kodu asynchronicznego, w szczególności – wykorzystującego funkcje *callback*.

Techniki te opisane są w kolejnych podrozdziałach.

try, catch

W języku JavaScript dostępne są instrukcje `try` i `catch`, działające podobnie jak w innych językach programowania. Przykładowo, w przypadku **synchronicznego fragmentu kodu** obsługę błędów można zrealizować w następujący sposób:

```
const fs = require('fs');

try {
  // Synchroniczne wczytanie pliku o nazwie "my_file.txt"
  const fileContents = fs.readFileSync("my_file.txt", "utf8");
  console.log("Plik został wczytany");
}
catch (error) {
  // Poniższe instrukcje zostaną wykonane, jeśli podczas wykonywania
  // bloku kodu poprzedzonego poleceniem "try" wystąpi błąd.
  console.error("Nie udało się wczytać pliku");
  console.error(`Opis błędu:\n${error}`);
}
```

`try`, `catch` + `async`, `await`

Z instrukcji `try`, `catch` można korzystać również w programach **asynchronicznych, używających *promises*, realizowanych za pomocą słów `async`, `await`**. W takim wypadku, kod zawarty w bloku `catch` zostanie wykonany, jeśli *promise* zostanie odrzucona (tzn. przejdzie w stan ***rejected***):

```
const { readFile } = require('fs').promises;

async function main() {
  try {
    // Asynchroniczne wczytanie pliku za pomocą promise
    const data = await readFile("my_file.txt", "utf8");

    // Poniższa instrukcja zostanie wykonana, jeśli
    // obietnica readFile zostanie spełniona (fulfilled)
    console.log(`Zawartość pliku:\n${data}`);
  }
  catch (rejectionReason) {
    // Poniższa instrukcja zostanie wykonana, jeśli
    // obietnica readFile zostanie odrzucona (rejected)
    console.error(`Nie udało się wczytać pliku, bo:\n${rejectionReason}`);
  }
}
```

Literatura

- MDN Web Docs: try...catch - JavaScript
- MDN Web Docs: *Promise* – metoda *catch*

process.on

Do obsługi błędów w asynchronicznych programach w Node.js, wykorzystujących funkcje *callback*, można używać **obiektu `process`:**

```
process.on('uncaughtException', err => {
  // kod znajdujący się w tym bloku zostanie wykonany,
  // gdy wystąpi nieprzechwycony wyjątek.

  // wypisanie treści błędu w kanale stderr
  console.error(`Uncaught Exception: ${err}`);

  // tu powinny nastąpić porządki:
  // zamykanie uchwytów do plików itp.

  // zakończenie działania programu
  process.exit(1)
})
```

Obiekt `process` jest instancją klasy `EventEmitter`, która służy do obsługi zdarzeń. **Metoda `on`**, dostępna w tej klasie, przyjmuje dwa argumenty wejściowe:

```
process.on([eventName], [listener])
```

Pierwszy z nich określa rodzaj zdarzenia, a drugi – funkcję, która ma być wywołana, gdy zdarzenie danego rodzaju będzie miało miejsce.

Zdarzenia `uncaughtException` mają miejsce:

- gdy w trakcie wykonywania programu występuje błąd JavaScript;
- w wyniku wywołania polecenia `throw`, np.:

```
throw new Error("[opis błędu]");
```

Metoda `exit` obiektu `process` powoduje zatrzymanie działania programu (ang. *terminate*) z zadanym kodem – w szczególności:

- kod `0` oznacza "sukces", tzn. zakończenie wykonywania programu zgodnie z planem;
- kod `1` oznacza zakończenie wykonywania programu w wyniku nieobsłużonego błędu.

We fragmencie kodu przytoczonym na początku niniejszego podrozdziału, jako reakcja na zdarzenia typu `uncaughtException` określona jest funkcja anonimowa, która:

- wypisuje treść błędu, otrzymaną jako argument wejściowy `err`;
- powinna wykonywać wszelkie porządki, które wymagane są przed zatrzymaniem wykonywania programu (np. zamknięcie otwartych uchwytów do plików);
- zatrzymuje działanie programu z kodem `1`.

Przykładowy program, w którym wykorzystane jest powyższe podejście:

```
const fs = require('fs');  // moduł służący do operacji na plikach

// Wczytanie pliku o nazwie "my_file.txt"
fs.readFile('my_file.txt', 'utf8', (err, data) => {

  if (err) {  // podczas próby wczytania pliku nastąpił błąd
    throw err;  // "wyrzucenie" błędu, czyli wywołanie zdarzenia uncaughtException
  }

  // wypisanie zawartości pliku w wierszu poleceń
  // (ma miejsce tylko, jeśli nie wystąpił błąd)
  console.log(data);
```

```
});

// Obsługa błędów
process.on('uncaughtException', err => {
  console.error(`Uncaught exception: ${err}`);
  process.exit(1)
})
```

Metodę `process.on` można również wykorzystać do obsługi odrzuconych *promises*. Należy wówczas „przechwytywać” zdarzenia `unhandledRejection`:

```
process.on('unhandledRejection', (reason, promise) => {
  console.error('Unhandled rejection at:', promise, 'reason:', reason);
});
```

Literatura

- Dokumentacja Node.js: *Process*
- Dokumentacja Node.js: *Process* – zdarzenia *uncaughtException*
- Dokumentacja Node.js: *EventEmitter* – metoda *on*

Operacje na plikach w Node.js

Moduł `fs` (od ang. akronimu *file system*, system plików) służy do wykonywania operacji na plikach i folderach.

```
const fs = require('fs');
```

W module `fs` dostępne jest **API `promises`**, które umożliwia asynchroniczne operacje na plikach z wykorzystaniem *promises*:

```
const fsPromises = require('fs').promises;
```

Wczytywanie plików

Do asynchronicznego wczytywania plików służy **funkcja `readFile`** z API `promises` modułu `fs`:

```
readFile([nazwa_pliku], [kodowanie])
```

Funkcja ta zwraca obiekt typu *promise*. Można jej użyć np. w następujący sposób:

```
const { readFile } = require('fs').promises;

readFile("my_file.txt", "utf-8").then(fileContents => {

  // Przetwarzanie treści pliku, zawartej w zmiennej fileContents:
  console.log(fileContents);

}).catch(rejectionReason => {

  // Poniższa instrukcja zostanie wywołana, jeśli
  // nie uda się wczytać pliku:
  console.error(rejectionReason);
});
```

Bez *promises*

W module `fs` jest jeszcze inna funkcja `readFile`, poza API `promises`, która również działa asynchronicznie, ale z wykorzystaniem funkcji *callback*:

```
readFile([nazwa_pliku], [kodowanie], [callback]);
```

Można jej używać np. w następujący sposób:

```
const fs = require('fs');

fs.readFile("my_file.txt", "utf-8", (err, fileContents) => {

  // "Wyrzucenie" błędu, który mógł wystąpić przy próbie wczytania pliku:
  if (err) throw err;

  // Przetwarzanie treści pliku, zawartej w zmiennej fileContents:
  console.log(fileContents);
});
```

`Buffer`

Jeśli nie poda się sposobu kodowania – drugiego argumentu funkcji `readFile` – otrzyma się zmienną klasy `Buffer`, czyli sekwencję bajtów. Wypisanie zawartości takiej zmiennej wprost za pomocą polecenia `console.log(data)` nie daje czytelnego efektu:

```
const { readFile } = require('fs').promises;

// Brak drugiego argumentu - sposobu kodowania:
readFile("my_file.txt").then(fileContents => {
  console.log(fileContents);
});
```

```
<Buffer 53 65 67 72 65 67 75 6a 20 c5 9b 6d 69 65 63 69>
```

Przetwarzanie ścieżek plików

W środowisku Node.js zawsze dostępne są **zmienne `__filename` i `__dirname`** (z dwoma podkreślnikami _ na początku). Pierwsza z nich przechowuje nazwę (razem z pełną ścieżką) pliku *.js* (modułu), który jest w danym momencie wykonywany. Druga przechowuje ścieżkę katalogu, w którym ten plik się znajduje.

```
console.log(__filename);
console.log(__dirname);
```

```
D:\Documents\node-tests\fs_test.js
D:\Documents\node-tests
```

Użycie tych zmiennych nie wymaga importowania żadnego modułu.

Moduł `path`

Do przetwarzania ścieżek plików i folderów (ang. ***path*** albo ***file path***) służy moduł `path`.

Jeśli potrzebna jest ścieżka złożona z kilku elementów, zamiast konstruować ją „ręcznie” lepiej jest użyć metody `join` z modułu `path`, ponieważ funkcja ta bierze pod uwagę zasady konstrukcji ścieżek specyficzne dla używanego systemu operacyjnego:

```
const path = require('path');
var filePath = path.join(__dirname, 'files', 'my_file.txt');
console.log(filePath);
```

```
D:\Documents\node-tests\files\my_file.txt
```

W module `path` są różne inne funkcje, służące do przetwarzania ścieżek:

```
console.log(path.dirname(filePath));
```

```
D:\Documents\node-tests\files
```

```
console.log(path.basename(filePath));
```

```
my_file.txt
```

```
console.log(path.extname(filePath));
```

```
.txt
```

```
// Funkcja parse zwraca obiekt, którego własności
// reprezentują różne informacje o ścieżce.
pathInfo = path.parse(filePath);

console.log(pathInfo.root);
console.log(pathInfo.dir);
console.log(pathInfo.base);
console.log(pathInfo.ext);
console.log(pathInfo.name);
```

```
D:\
D:\Documents\node-tests\files
my_file.txt
.txt
my_file
```

Tworzenie folderów

Do synchronicznego tworzenia folderów służy **funkcja `mkdirSync`** z modułu `fs`:

```
const fs = require('fs');
const path = require('path');

// Ścieżka folderu, który chcemy stworzyć:
var dirPath = path.join(__dirname, "new_dir");

try {
  fs.mkdirSync(dirPath);
}
catch(err) {
  // Folderu nie udało się stworzyć
  // (np. dlatego, że już wcześniej istniał)
  console.error(err);
}
```

Do asynchronicznego tworzenia folderów można używać **funkcji `mkdir`** z API `promises` modułu `fs`:

```
const fsPromises = require('fs').promises;
const path = require('path');

// Ścieżka folderu, który chcemy stworzyć:
var dirPath = path.join(__dirname, "new_dir");

fsPromises.mkdir(dirPath).then(() => {
  console.log(`Folder ${dirPath} został stworzony.`);

}).catch(rejectionReason => {
  // Folderu nie udało się stworzyć
  // (np. dlatego, że już wcześniej istniał)
  console.error(rejectionReason);
});
```

Sprawdzanie, czy dany folder istnieje

Do synchronicznego sprawdzenia, czy folder o zadanej ścieżce istnieje, służy **funkcja `existsSync`** modułu `fs`:

```
if (fs.existsSync(dirPath)) {
  console.log(`Folder ${dirPath} istnieje.`);
}
```

Do asynchronicznego sprawdzenia, czy folder o zadanej ścieżce istnieje, można wykorzystać **funkcję `access`** z API `promises` modułu `fs`. Jeśli folder nie istnieje, obietnica `access` zostanie odrzucona:

```
try {
  await fsPromises.access(dirPath);
} catch {
  console.log(`Folder ${dirPath} nie istnieje.`);
}
```

Przykładowy program, w którym chcielibyśmy asynchronicznie:

- określić ścieżkę folderu, w którym zamierzamy przechowywać pliki,

- sprawdzić, czy ten folder istnieje,
- jeśli nie istnieje – stworzyć go,
- kontynuować pracę z plikami, korzystając z tego folderu,

mógłby wyglądać następująco:

```
const fsPromises = require('fs').promises;
const path = require('path');

// Nazwa folderu z plikami:
var filesDirName = "files";

// Cała ścieżka folderu z plikami:
var filesDirPath = path.join(__dirname, filesDirName);

async function main() {

  // Sprawdzenie, czy folder "files" istnieje:
  try {
    await fsPromises.access(filesDirPath);
  } catch {
    // Folder "files" nie istnieje, więc go stwórzmy:
    await fsPromises.mkdir(filesDirPath);
  }

  // ... dalsze operacje, wykorzystujące folder "files"
}

main()
```

Tworzenie i modyfikacja plików

Tworzenie nowych plików

Do asynchronicznego tworzenia plików służy **funkcja `writeFile`** z API `promises` modułu `fs`:

```
const fsPromises = require('fs').promises;
const path = require('path');

// Nazwa i ścieżka pliku, który chcemy stworzyć:
var fileName = "my_file.txt";
var filePath = path.join(__dirname, fileName);

async function main() {
  try {
    await fsPromises.writeFile(filePath, "All work and no play makes Jack a dull boy\n");
    console.log("File created");
  }
  catch (rejectionReason) {
    console.error(rejectionReason);
  }
}

main();
```

Jeśli plik o zadanej ścieżce nie istnieje, funkcja `writeFile` go stworzy; jeśli istnieje, jego wcześniejsza zawartość zostanie usunięta. Funkcja `writeFile` **nie stworzy jednak folderu**; jeśli folder określony w zadanej ścieżce nie istnieje, zapis pliku nie powiedzie się.

Dopisywanie do istniejących plików

Aby dopisać treść do istniejącego pliku, nie usuwając jego wcześniejszej zawartości, można wykorzystać **funkcję `appendFile`** z API `promises` modułu `fs`:

```
for (let n = 0; n < 999; n++) {
  await fsPromises.appendFile(filePath, "All work and no play makes Jack a dull boy\n");
}
```

Podobnie jak `writeFile`, funkcja `appendFile` stworzy nowy plik, jeśli plik o podanej ścieżce nie istnieje (ale nie stworzy folderu).

Zmiana nazw, przenoszenie i usuwanie plików

Funkcja `rename` służy do przenoszenia albo zmiany nazwy pliku:

```
await fsPromises.rename([stara_ścieżka], [nowa_ścieżka]);
```

Jeśli plik o docelowej ścieżce `[nowa_ścieżka]` istnieje, zostanie nadpisany.

Do usuwania plików służy **funkcja `unlink`**:

```
await fsPromises.unlink([ścieżka_pliku_do_usunięcia]);
```

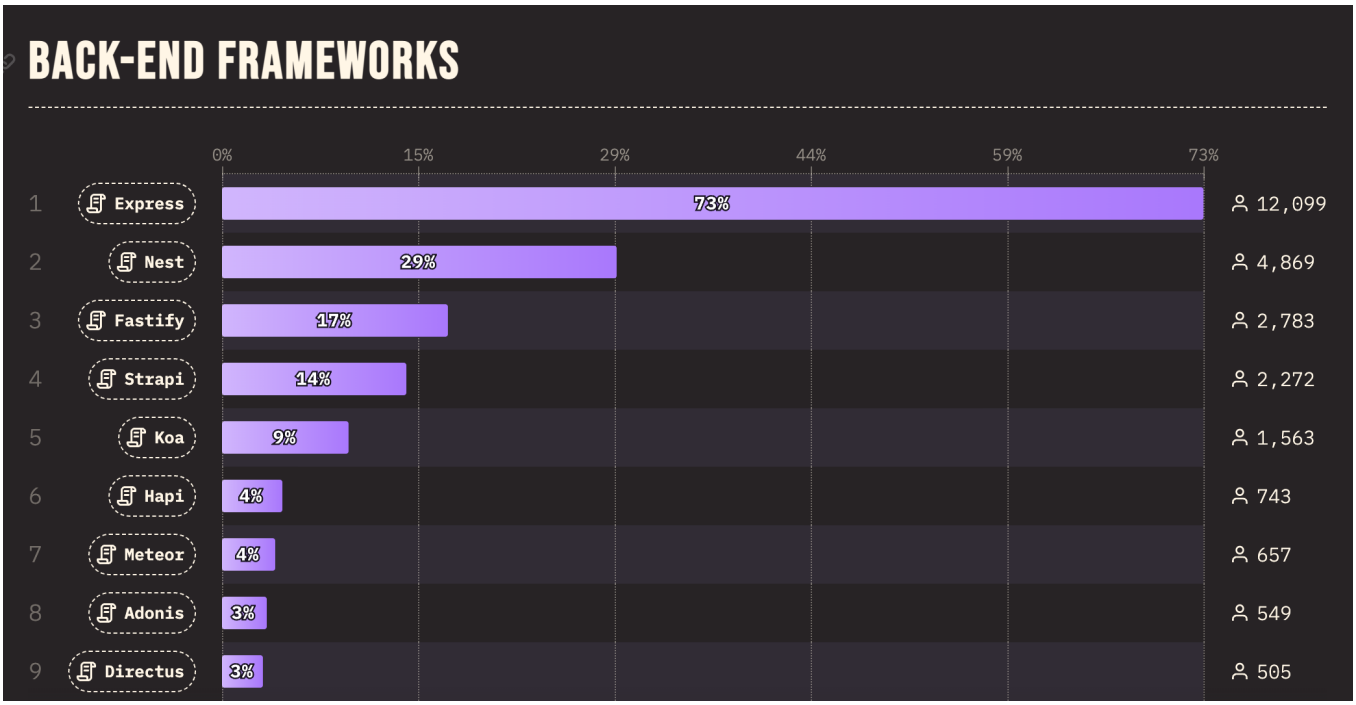
Express

Express jest narzędziem, które ułatwia tworzenie aplikacji serwerowych za pomocą Node.js.

https://expressjs.com/

Da się z powodzeniem tworzyć aplikacje serwerowe, korzystając jedynie z „surowego" Node.js, jednak dostępne darmowo platformy programistyczne (ang. *frameworks*) pozwalają czynić to znacznie szybciej i prościej. Express jest najpopularniejszą spośród takich platform.

Statystyki wykorzystania *back-endowych* platform programistycznych przeznaczonych dla języka JavaScript w 2023 roku:



źródło: State of JavaScript 2023

Inicjalizacja Express

Express instaluje się osobno dla poszczególnych projektów. Aby móc z niego korzystać, trzeba mieć zainstalowane środowisko Node.js i działający dla danego projektu system npm. W przypadku nowego projektu, należy otworzyć wiersz poleceń w odpowiednim folderze i zainicjalizować npm:

```
> npm init
```

Express można zainstalować dla danego projektu za pomocą npm:

```
> npm install express
```

W aplikacjach tworzonych za pomocą Express główny plik typowo nazywa się *app.js* (ale można użyć innej nazwy). Można tę nazwę ustawić podczas inicjalizacji systemu npm (jako *entry point*), albo później, „ręcznie” zmieniając wartość własności `"main"` w pliku *package.json*.

Ponadto, warto zdefiniować skrypty, służące do uruchamiania aplikacji, zarówno w docelowym trybie, jak i – na czas pracy nad aplikacją – w trybie *watch*:

Plik *package.json*

```
{
  ...
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "dev": "node --watch app.js"
  },
  ...
}
```

Szkielet aplikacji

Aby zacząć pracę z platformą Express, w głównym pliku aplikacji należy ją zaimportować za pomocą polecenia `require`:

```
const express = require('express');
```

Za pomocą polecenia `express()` należy następnie jednorazowo, dla całej aplikacji, stworzyć obiekt stanowiący „instancję serwera Express", nazywany typowo `app`:

```
const app = express();
```

Taki serwer można uruchomić, korzystając z metody `listen`:

```
app.listen([numer_portu], [funkcja_wywołana_po_uruchomieniu]);
```

Po wywołaniu metody `listen` uruchomiona aplikacja serwerowa będzie dostępna pod podanym numerem portu. Jako drugi argument metody `listen` podaje się zwykle funkcję, która w wierszu poleceń wypisze komunikat o działaniu aplikacji:

```
const port = 3000;
app.listen(port, () => console.log(`Aplikacja serwerowa działa na porcie ${port}.`));
```

Aby aplikacja była w stanie generować użyteczne odpowiedzi na żądania, między poleceniem tworzącym obiekt `app` i wywołaniem jego metody `listen` powinien się znaleźć m.in. kod realizujący trasowanie. Jest to omówione w dalszych podrozdziałach.

Główny plik aplikacji może więc mieć następującą postać:

Plik *app.js*

```
const express = require('express');
const app = express();
const port = 3000;

// ...
// tu powinien się znaleźć m.in. kod realizujący trasowanie
// ...

app.listen(port, () => console.log(`Aplikacja serwerowa działa na porcie ${port}.`));
```

Trasowanie

Obiekt `app`, utworzony za pomocą polecenia `express()`, dysponuje metodami odpowiadającymi metodom HTTP: `get`, `post`, `put`, `delete` itd. Za pomocą tych metod można określać trasy, które będą obsługiwane przez aplikację serwerową. Każda z tych metod wymaga podania co najmniej dwóch argumentów wejściowych:

- ścieżki (ang. *path*),
- funkcji (nazywanej po angielsku *route handler*, co nie ma powszechnie przyjętego polskiego odpowiednika) wywoływanej po nadejściu żądania zawierającego daną ścieżkę i metodę HTTP.

```
app = express();
app.[metoda]([ścieżka], [route_handler]);
```

Jeśli, przykładowo, chcemy, aby żądanie metodą GET dla ścieżki */hello* poskutkowało wysłaniem w odpowiedzi tekstu *"Hello, world"*, to można to zrealizować w następujący sposób:

```
app.get('/hello', (req, res) => {
  res.send('Hello, world');
});
```

W najprostszym przypadku, funkcja stanowiąca *route handler* powinna przyjmować dwa argumenty wejściowe, typowo nazywane `req` i `res` (od ang. *request* i *response*):

- pierwszy argument wejściowy reprezentuje przychodzące żądanie,
- drugi argument wejściowy reprezentuje odpowiedź, która zostanie wysłana przez aplikację serwerową.

Określanie ścieżek

Ścieżka, stanowiąca pierwszy argument metody `app.get`, `app.post` itp., może być podana w formie wyrażenia regularnego (ang. *regular expression*). Przykładowo, poniższe polecenie:

```
app.get('^/$|/index(.html)?', handler);
```

przypisze funkcję `handler` do wszystkich trzech następujących ścieżek:

- */*
- */index*
- */index.html*

Trasy, zdefiniowane w kodzie aplikacji serwerowej, są podczas trasowania sprawdzane **po kolei**, a gdy jakaś trasa okaże się pasować do żądania, to dalsze nie będą już sprawdzane. Na końcu pliku można więc umieścić kod obsługujący wszystkie nieprzewidziane trasy:

```
// ...
// obsługa różnych przewidzianych tras
// ...

app.get("/*", (req, res) => unexpected_route_handler);
```

"`/*`" oznacza tu dowolną ścieżkę zaczynającą się od ukośnika.

Konstruowanie odpowiedzi

Odpowiedzi na żądania należy konstruować i wysyłać za pomocą metod obiektu `res` (będącego jednym z argumentów wejściowych metod takich jak `app.get`, `app.post`, ...). Do metod generujących odpowiedzi należą m.in.:

- `res.send`, służąca do wysyłania różnego rodzaju odpowiedzi, w tym prostego tekstu;
- `res.sendFile`, służąca do wysyłania plików;

- `res.redirect`, służąca do przekierowywania żądań;
- `res.download`, służąca do zlecania pobrania pliku.

Wywołanie dowolnej z powyższych metod skutkuje wysłaniem odpowiedzi i zakończeniem przetwarzania żądania. Jeśli żadna taka metoda nie zostanie wywołana, to żądanie może pozostać bez odpowiedzi.

Przykładowo, jeśli w odpowiedzi na żądanie GET ze ścieżką */contact* ma być przesłany plik *contact.html*, zapisany w folderze *views*, to można to zrealizować w następujący sposób:

```
const path = require('path')

// ...

app.get("/contact", (req, res) => {
  res.sendFile(path.join(__dirname, "views", "contact.html"));
});
```

Kody odpowiedzi

Express automatycznie dobiera kody odpowiedzi (np. 200 – *OK*, 404 – *Not Found* itp.). Można jednak również samemu ustalić kod za pomocą metody `res.status`. Przykładowo, jeśli w odpowiedzi na nieprzewidzianą ścieżkę chcemy wysłać przygotowany specjalnie na taką okazję plik *not_found.html*, zapisany w folderze *views*, to powinniśmy opatrzyć go kodem 404; można to zrealizować w następujący sposób:

```
res.status(404).sendFile(path.join(__dirname, 'views', 'not_found.html'));
```

Przekierowanie za pomocą metody `res.redirect` domyślnie opatrzone jest kodem 302 (oznaczającym przekierowanie tymczasowe). Przekierowanie z kodem 301 (sygnalizującym, że adres został zmieniony na stałe) można zrealizować w następujący sposób:

```
res.redirect(301, [nowy_adres]);
```

Przykład

Poniższy fragment kodu stanowi zebranie wcześniejszych przykładów i implementuje prostą aplikację serwerową. Aplikacja ta:

- W odpowiedzi na żądanie GET pod dowolną ze ścieżek: */*, */index* lub */index.html* wysyła plik *index.html*, zapisany w folderze *views*, z kodem 200;
- W odpowiedzi na wszelkie inne żądania GET wysyła plik *not_found.html*, zapisany w folderze *views*, z kodem 404.

```
const path = require('path')
const express = require('express');
const app = express();
const port = 3000;

app.get("^/$|/index(.html)?", (req, res) => {
  res.sendFile(path.join(__dirname, "views", "index.html"));
});

app.get("/*", (req, res) => {
  res.status(404).sendFile(path.join(__dirname, "views", "not_found.html"));
})

app.listen(port, () => console.log(`Aplikacja serwerowa działa na porcie ${port}.`));
```

Więcej informacji:

- Dokumentacja Express – *Routing*

Pliki statyczne

W typowej sytuacji, oprócz plików z kodem JavaScript, które implementują podstawowe funkcje aplikacji serwerowej, oraz plików HTML, wysyłanych w odpowiedzi na żądania, potrzebne są również inne pliki: obrazki, arkusze stylów CSS itp., zwane często plikami „statycznymi” (ang. ***static files***). Odwołania do tych plików mogą znajdować się w wysyłanych plikach HTML, jednak trzeba dodatkowo zadbać o to, by faktycznie trafiły one do klienta. Służy do tego tzw. *middleware* `express.static`. Aby go użyć, należy wywołać metodę `app.use`, przed definicjami tras, w następujący sposób:

```
const express = require('express');
const app = express();

app.use(express.static([ścieżka_folderu_z_publicznymi_plikami_statycznymi]));

// definicje tras: app.get(...), app.post(...) itd.
// ...
```

Express będzie wówczas poszukiwał plików w ścieżkach określonych względem podanego folderu z publicznymi plikami statycznymi. Typowo takie pliki zapisuje się w folderze *public* (i jego podfolderach). Zakładając, że pliki aplikacji serwerowej zorganizowane są w następujący sposób:

```
├── app.js
├── package.json
├── public
│   └── stylesheets
│       └── style.css
└── views
    ├── index.html
    └── not_found.html
```

a plik *index.html* ma zawierać odwołanie do pliku *style.css*, należy to zrealizować w następujący sposób:

Plik app.js

```
const path = require('path')

...

app.use(express.static(path.join(__dirname, 'public')));

...
```

Plik index.html

```
...

<link rel="stylesheet" type="text/css" href="stylesheets/style.css" />

...
```

Mimo że w powyższym przykładzie ścieżka z pliku *index.html* do pliku *style.css* wydaje się wyglądać następująco:

`"../public/stylesheets/style.css"`

to należy w pliku *index.html* podać ścieżkę względem folderu *public*:

`"stylesheets/style.css"`

ponieważ w pliku *app.js* Express został poinstruowany, by tam właśnie szukać plików statycznych.

Więcej informacji:

- Dokumentacja Express – *Static files*

Testowanie aplikacji

Aby testować aplikację lokalnie, należy ją uruchomić, np. wpisując w wierszu poleceń (jeśli główny plik nazywa się *app.js*):

```
> node --watch app.js
```

albo, jeśli odpowiedni skrypt został zdefiniowany w pliku *package.json*:

```
> npm run dev
```

Jeśli został ustawiony numer portu 3000, to uruchomiona aplikacja będzie dostępna po wpisaniu w przeglądarce internetowej adresu:

localhost:3000

Wpisanie powyższego adresu w przeglądarce poskutkuje wysłaniem do aplikacji serwerowej żądania GET ze ścieżką /. Żądania pod inne ścieżki można generować, dopisując te ścieżki na końcu adresu, np.: localhost:3000/index (ścieżką jest wówczas */index*).

Jeśli wykorzystany jest tryb *watch*, to zmiany wprowadzane w kodzie aplikacji będą realizowane bez potrzeby ręcznego jej restartowania. Będą one widoczne w przeglądarce po odświeżeniu strony.

Bazy danych

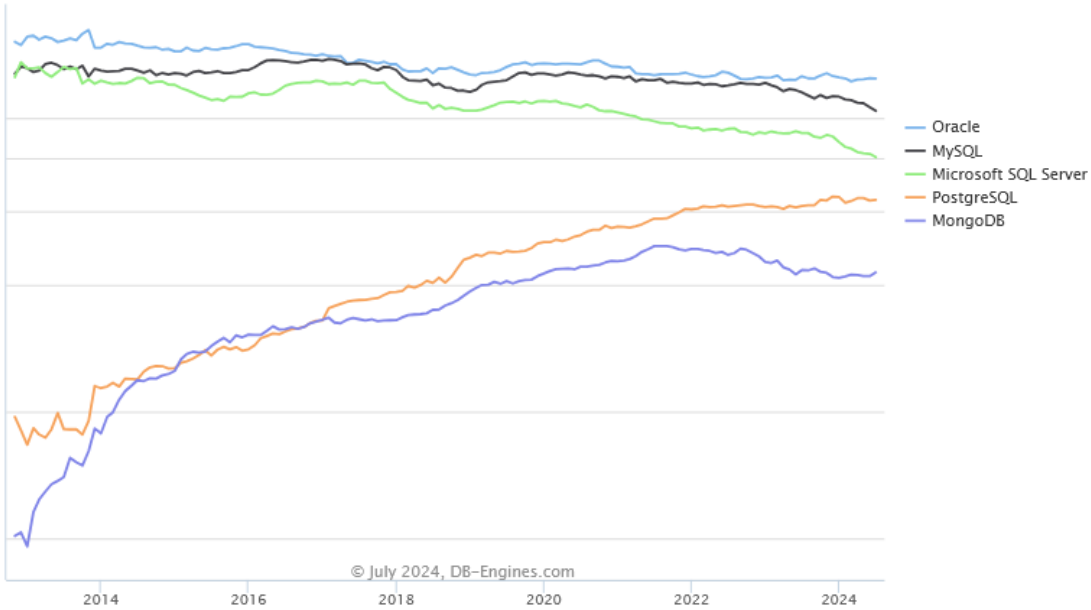
Istnieją różne sposoby przechowywania danych oraz liczne, dostępne zarówno darmowo, jak i odpłatnie, systemy zarządzania tymi danymi. Bazy danych (ang. *databases*) można podzielić na:

- relacyjne (ang. *relational*),
- nierelacyjne (ang. *non-relational*).

Relacyjne bazy danych przeważnie obsługiwane są za pomocą **języka SQL**. Dane w takich bazach są zorganizowane w formie tabel. W porównaniu z innymi typami baz danych, bazy relacyjne charakteryzują się większą „sztywnością" struktur, w których przechowywane są dane. Są one od dawna bardzo powszechne.

Nierelacyjne bazy danych (nazywane niekiedy bazami ***NoSQL***) realizowane są różnorako. Dane mogą w nich być przechowywane np. w postaci par klucz-wartość (ang. *key-value*) lub tzw. dokumentów (ang. *documents*). Takie bazy dopuszczają większą swobodę w organizacji danych niż bazy relacyjne, a gdy danych jest bardzo dużo, są one również bardziej wydajne. Od pewnego czasu zyskują coraz większą popularność.

Ranking popularności systemów zarządzania bazami danych (5 najpopularniejszych):



źródło: DB-Engines

Na powyższym obrazku przedstawiono oceny popularności 5 systemów zarządzania bazami danych, najpowszechniej stosowanych w czerwcu 2024 r:

- Cztery z nich (**Oracle**, **MySQL**, **Microsoft SQL Server** i **PostgreSQL**) służą do zarządzania relacyjnymi bazami danych.
- **MongoDB** służy do zarządzania „dokumentowymi", nierelacyjnymi bazami danych.

Obsługa baz danych wymaga zastosowania technik programowania **asynchronicznego** – dlatego, że każda operacja wymagająca komunikacji z bazą danych (lub przetwarzaniem pozyskanych z niej danych) może długo trwać, co w programie synchronicznym zablokowałoby pętlę zdarzeń.

W następnych podrozdziałach omówione są pokrótce dwa przykładowe systemy zarządzania bazami danych – MySQL i MongoDB – z uwzględnieniem możliwości ich obsługi za pomocą środowiska Node.js.

Podstawy SQL

Język SQL służy do obsługi relacyjnych baz danych. Dane w bazach relacyjnych są zorganizowane w postaci tabel, w których z góry określona jest zawartość poszczególnych kolumn, a także format, w jakim ta zawartość ma być zapisywana. Poszczególne wiersze tabeli reprezentują poszczególne wpisy do bazy.

Poniższa tabela o nazwie `JBS` zawiera dane kilku (wybranych arbitralnie) muzyków, którzy grywali w zespołach Jamesa Browna. Każdy jej wiersz odpowiada innemu muzykowi, a poszczególne kolumny: ich numerom identyfikacyjnym (`ID`), utworzonym na potrzeby tej przykładowej bazy danych, nazwiskom (`NAME`) i instrumentom, na których grali (`INSTRUMENT`).

Tabela JBS

ID	NAME	INSTRUMENT
1	Jimmy Nolen	gitara
2	Maceo Parker	saksofon
3	Bernard Purdie	perkusja
4	William Collins	gitara basowa
5	John Starks	perkusja
6	Fred Wesley	puzon

Język SQL pozwala zarządzać bazami danych, pozyskiwać dane z tabel oraz wykonywać na nich różne operacje za pomocą poleceń zwanych „zapytaniami” (ang. *queries*). Stworzenie bazy danych o nazwie `jb_db` realizowane jest przez następujące polecenie:

```
CREATE DATABASE IF NOT EXISTS jb_db
```

przy czym fragment `IF NOT EXISTS` gwarantuje, że jeśli baza danych o podanej nazwie (`jb_db`) już istnieje, to nic specjalnego się nie wydarzy.

Po stworzeniu bazy danych można się do niej „podłączyć” (tj. zacząć nią zarządzać), korzystając z polecenia USE:

```
USE jb_db
```

Utworzenie pustej tabeli o strukturze takiej, jaką ma powyższa tabela `JBS`, realizowane jest przez następujące polecenie:

```
CREATE TABLE IF NOT EXISTS JBS(ID INT PRIMARY KEY AUTO_INCREMENT, NAME VARCHAR(255), INSTRUMENT VARCHAR(255))
```

przy czym:

- fragment `ID INT` oznacza, że utworzona zostanie kolumna o nazwie `ID`, zawierająca liczby całkowite (`INT`);
- fragment `PRIMARY KEY` oznacza, że wartości w kolumnie `ID` mogą służyć za unikalne identyfikatory poszczególnych wierszy;
- fragment `AUTO_INCREMENT` oznacza, że przy tworzeniu każdego nowego wiersza automatycznie zostanie do niego przypisana unikalna wartość `ID` (bez potrzeby ustawiania jej „ręcznie”);
- fragment `NAME VARCHAR(255), INSTRUMENT VARCHAR(255)` oznacza, że utworzone zostaną kolumny o nazwach `NAME` i `INSTRUMENT`, zawierające ciągi znaków (czyli dane tekstowe), których długość nie będzie przekraczać 255 bajtów.

Odczytanie z powyższej tabeli nazwisk wszystkich perkusistów realizowane jest przez następujące zapytanie:

```
SELECT NAME FROM JBS WHERE INSTRUMENT = "perkusja"
```

Wstawienie do tabeli nowego wiersza:

```
INSERT INTO JBS(NAME, INSTRUMENT) VALUES ("Tyrone Jefferson", "puzon")
```

Zmiana wybranych wartości:

```
UPDATE JBS SET NAME = "Bootsy Collins" WHERE NAME = "William Collins"
```

Usunięcie wybranego wiersza:

```
DELETE FROM JBS WHERE ID = 4
```

Usunięcie wszystkich danych z tabeli:

```
TRUNCATE JBS
```

Ataki SQL injection

Działanie aplikacji serwerowej często wymaga formułowania zapytań SQL, zawierających elementy wprowadzane przez użytkowników aplikacji. Należy wówczas zachować ostrożność – robiąc to niefrasobliwie, można narazić bazę danych na ataki zwane ***SQL injection***. Takim atakom stosunkowo łatwo jest zaradzić, niemniej należy o tym pamiętać. W internecie – m.in. na Wikipedii – można znaleźć liczne opracowania tego tematu.

The SQL Murder Mystery

Pod poniższym adresem można znaleźć materiał do samodzielnego ćwiczenia obsługi języka SQL w formie zagadki kryminalnej:

https://mystery.knightlab.com/

Konfiguracja MySQL

MySQL to system zarządzania relacyjnymi bazami danych za pomocą języka SQL. Aby móc z niego korzystać, należy zainstalować i skonfigurować oprogramowanie dostępne pod adresem:

MySQL Community Downloads

> W ramach zarządzania bazami danych za pomocą systemu MySQL rejestruje się „użytkowników" (ang. ***users***) i nadaje im różne uprawnienia. Bezpośrednio po instalacji oprogramowania MySQL zarejestrowany jest użytkownik **`root`**, dysponujący pełnymi uprawnieniami.
>
> Innym użytkownikom, rejestrowanym na potrzeby rozwijanych aplikacji, należy nadawać minimalne niezbędne zestawy uprawnień; w szczególności, należy unikać dawania osobom korzystającym z aplikacji ryzykownych możliwości, które ma użytkownik `root` (takich jak odczytywanie lub usuwanie wszystkich danych w bazie).

W celu konfiguracji systemu MySQL należy m.in.:

- określić port, pod którym serwer będzie udostępniał funkcje systemu MySQL (domyślnie: 3306);
- ustawić hasło dla użytkownika `root`.

Aby było możliwe korzystanie z systemu MySQL, musi być uruchomiony tzw. serwer MySQL (ang. *MySQL server*). Bezpośrednio po instalacji oprogramowania MySQL może on być uruchomiony automatycznie, natomiast do ręcznego uruchamiania serwera MySQL służy program `mysqld`. Pierwsze uruchomienie serwera może też wymagać dodatkowych czynności inicjalizacyjnych. Więcej informacji na ten temat można znaleźć w dokumentacji oprogramowania MySQL:

- Installing MySQL
- Postinstallation Setup and Testing

Częścią oprogramowania MySQL jest program `mysql`, obsługiwany w wierszu poleceń, umożliwiający wykonywanie poleceń w języku SQL. Aby go uruchomić, łącząc się z lokalnym serwerem (`localhost`) jako użytkowik `root`, należy w wierszu poleceń wywołać następujące polecenie:

```
> mysql -h localhost -u root -p
```

(zakładając, że program `mysql` jest „widoczny", np. w systemie Windows – że odpowiednia ścieżka jest dodana do zmiennej środowiskowej *Path*.)

W powyższym poleceniu:

- `-h` (skrót od `host`) poprzedza nazwę serwera;
- `-u` (skrót od `user`) poprzedza nazwę użytkownika;
- `-p` (skrót od `password`) sygnalizuje, że uwierzytelnienie użytkownika ma się odbyć poprzez podanie hasła, ale hasło to podaje się po wywołaniu powyższego polecenia (nie jako kolejny argument wejściowy).

Jeśli istnieje już jakaś baza danych (np. o nazwie `jb_db`) to można jej nazwę podać jako argument przy uruchomieniu programu `mysql`, aby od razu się do niej podłączyć:

```
> mysql -h localhost -u root -p jb_db
```

> Wbrew pozorom, w powyższym poleceniu `jb_db`, następujące po `-p`, nie zostanie zinterpretowane jako hasło, tylko jako nazwa bazy danych. Pytanie o hasło pojawi się dopiero po wywołaniu powyższego polecenia.

Po uruchomieniu programu `mysql` można wykonywać różne polecenia w języku SQL:

```
mysql> CREATE DATABASE IF NOT EXISTS jb_db;
```

> Program `mysql` wymaga zakończenia każdego polecenia średnikiem. Gdy zabraknie średnika, program będzie oczekiwał kontynuacji polecenia w kolejnym wierszu (co może ułatwić wpisywanie długich poleceń).

```
mysql> USE jb_db;
```

```
mysql> CREATE TABLE IF NOT EXISTS JBS(ID INT PRIMARY KEY AUTO_INCREMENT, NAME VARCHAR(255), INSTRUMENT
VARCHAR(255));
```

```
mysql> INSERT INTO JBS(NAME, INSTRUMENT) VALUES ("Jimmy Nolen", "gitara"), ("Maceo Parker", "saksofon");
```

```
mysql> SELECT * FROM JBS;
```

Darmowe oprogramowanie phpMyAdmin umożliwia wykonywanie operacji na bazach danych MySQL (tworzenie tabel, wyświetlanie i modyfikowanie ich zawartości itp.) za pomocą graficznego interfejsu użytkownika. W różnych systemach operacyjnych jego użycie może jednak wymagać instalacji dodatkowego oprogramowania.

Więcej informacji:

- Dokumentacja MySQL

Obsługa MySQL za pomocą Node.js

Do obsługi systemu MySQL za pomocą środowiska Node.js służy pakiet *mysql*, który można zainstalować za pomocą npm:

```
> npm install mysql
```

W kodzie JavaScript należy ten pakiet zaimportować:

```
const mysql = require('mysql');
```

Do przygotowania połączenia z systemem MySQL służy metoda `mysql.createConnection`. Jako jej argument wejściowy należy podać obiekt zawierający informacje dotyczące serwera i użytkownika, a także – opcjonalnie – nazwę bazy danych (w poniższym przykładzie – `'jb_db'`):

```
const connection = mysql.createConnection({
  host     : 'localhost',
  port     : 3306,
  user     : 'root',
  password : 'hasło',
  database : 'jb_db'
});
```

Jeśli przy wywołaniu metody `createConnection` nie poda się nazwy bazy danych, to potem będzie można się do wybranej bazy „podłączyć" za pomocą polecenia `USE` w języku SQL.

Aby nawiązać połączenie, należy wywołać metodę `connect` obiektu utworzonego za pomocą `mysql.createConnection`. Argumentem wejściowym metody `connect` jest funkcja *callback*, która zostanie wywołana zaraz po tym, jak połączenie zostanie nawiązane. Argumentem wejściowym tej funkcji *callback* jest natomiast obiekt – typowo nazywany `err` – reprezentujący ewentualny błąd zaistniały przy próbie nawiązania połączenia. Jeśli żaden błąd nie wystąpi, to `err` będzie miał wartość `null`. Do zakończenia połączenia służy metoda `end`.

W czasie powstawania niniejszych materiałów pakiet `mysql` przeznaczony dla Node.js nie jest w pełni zgodny z zabezpieczeniami stosowanymi w najnowszej wersji oprogramowania MySQL, przez co próba nawiązania połączenia może się zakończyć porażką. Szczegóły dotyczące tego problemu oraz sposoby jego rozwiązania można znaleźć na forum Stack Overflow:

https://stackoverflow.com/q/50093144

```
connection.connect((err) => {

  if (err) {
    console.error(`Błąd przy próbie nawiązania połączenia:\n\t${err.message}`);
    connection.end();
    return;
  }

  console.log("Nawiązano połączenie z bazą danych.");

  // tu należałoby jakoś wykorzystać nawiązane połączenie

  connection.end();
});
```

Funkcje z pakietu `mysql`, takie jak `connect`, działają asynchronicznie – przypisane do nich funkcje *callback* zostaną wywoływane w nieokreślonym momencie, ale oczekiwanie na ich wywołanie nie zablokuje pętli zdarzeń.

Po nawiązaniu połączenia można korzystać z bazy danych. Służy do tego metoda `query` obiektu utworzonego za pomocą `mysql.createConnection`. Za jej pomocą można wykonywać zapytania w języku SQL. W najprostszym przypadku przyjmuje ona dwa

argumenty wejściowe:

- zmienną tekstową, zawierającą zapytanie w języku SQL;
- funkcję *callback* do uruchomienia po wykonaniu ww. zapytania.

Funkcja *callback* metody `query` przyjmuje trzy argumenty wejściowe, nazywane typowo `err`, `results` i `fields`, zawierające odpowiednio:

- opis ewentualnego błędu, zaistniałego przy próbie wykonania zapytania, lub – w razie powodzenia – wartość `null`;
- wyniki zapytania;
- informacje o odczytanych kolumnach tabeli.

Poniższy fragment kodu umożliwia pozyskanie z tabeli `JBS` listy wszystkich perkusistów. Metoda `query` jest wywoływana wewnątrz funkcji *callback* metody `connect` – dzięki temu jest pewne, że zapytanie zostanie wysłane dopiero po nawiązaniu połączenia.

```
let sqlStatement = 'SELECT * FROM JBS WHERE INSTRUMENT = "perkusja"';

connection.connect((err) => {
  if (err) {
    console.error(`Błąd przy próbie nawiązania połączenia:\n\t${err.message}`);
    connection.end();
    return;
  }

  connection.query(sqlStatement, (err, results, fields) => {
    if (err) {
      console.error(`Błąd przy próbie wykonania zapytania SQL:\n\t${err.message}`);
      connection.end();
      return;
    }

    for (let row of results) {
      console.log(row.NAME);
    }

    connection.end();
  });
});
```

Za pomocą metody `query` można zrobić m.in. wszystko to, co w poprzednim podrozdziale zostało opisane w kontekście programu `mysql`. W szczególności, można, korzystając z niej, przygotować skrypt w języku JavaScript, przeznaczony do jednokrotnego uruchomienia, który posłuży do stworzenia bazy danych i pustych tabel, a także wstępnego wypełnienia ich danymi itd. Miałby to być, oczywiście, skrypt oddzielony od kodu aplikacji serwerowej, która późniejsze korzystanie z tych tabel umożliwiałaby docelowym użytkownikom.

Więcej informacji:

- Dokumentacja mysql (pakietu Node.js)

Podstawy MongoDB

MongoDB jest systemem zarządzania **nierelacyjnymi, dokumentowymi bazami danych**.

W bazach dokumentowych dane przechowywane są w postaci tzw. **dokumentów** (ang. *documents*), które mogą być zapisane w formacie takim jak np. **JSON** lub **XML**. W systemie MongoDB wykorzystywany jest format zbliżony do JSON.

Dokumenty

Pojedynczy **dokument** w dokumentowej bazie danych odpowiada, z grubsza, pojedynczemu **wierszowi tabeli** w bazie relacyjnej. Dokumentami w bazach MongoDB rządzą następujące prawa:

- Różne dokumenty (w ramach pojedynczej bazy) **mogą mieć różną strukturę**, a więc zawierać inny zestaw informacji – w przeciwieństwie do wierszy tabeli w bazie relacyjnej, które muszą się składać (w ramach pojedynczej tabeli) wszystkie z takich samych elementów.
- Każdy dokument zawiera tylko te informacje, które chcemy, żeby zawierał, i żadnych więcej – w przeciwieństwie do wierszy tabeli w bazie relacyjnej, dla których z góry określony jest zestaw kolumn do wypełnienia.
- Wszystkie dokumenty mogą być umieszczone w pojedynczej bazie danych „luzem”, tj. bez porządkowania ich w formie żadnych dodatkowych struktur. MongoDB umożliwia jednak grupowanie dokumentów w postaci tzw. **kolekcji** (ang. *collections*).
- Gdy potrzebne jest kilka „egzemplarzy” danego rodzaju informacji (np. gdy ktoś ma dwa numery telefonu), można w ramach pojedynczego dokumentu stworzyć **tablicę** (ang. *array*).

Poniższa ilustracja prezentuje przykładową bazę danych MongoDB, zawierającą m.in. informacje o kilku zespołach muzycznych.



Można zwrócić uwagę, że:

- Pojedyncza baza danych składa się z jednej lub więcej *kolekcji*, a każda kolekcja zawiera różne dokumenty.
- Każdy z dokumentów, reprezentujących poszczególne zespoły, ma m.in. własności `name` oraz `formed`, ale poza tym każdy zawiera trochę inny zestaw informacji, zależny od instrumentarium zespołu. Skoro w zespole *Fearless Flyers* nie ma wokalisty, to w odpowiadającym mu dokumencie nie ma potrzeby uwzględniania własności `vocal`. Inaczej wyglądałaby sytuacja w relacyjnej bazie danych: gdyby w tabeli była kolumna *vocal*, to w każdym wierszu trzeba byłoby do niej przypisać jakąś wartość (w razie braku wokalisty mogłaby to być np. wartość

`NULL`).
- Fakt, że w zespole *Fearless Flyers* jest dwóch gitarzystów, odwzorowany jest w taki sposób, że do własności `guitar` przypisana jest dwuelementowa tablica.

Pojedyncza baza danych może przechowywać wiele różnych, nie powiązanych ze sobą kolekcji i dokumentów. Zaleca się jednak, żeby przynajmniej dla każdego odrębnego projektu programistycznego tworzyć osobną bazę danych.

Identyfikatory dokumentów

Każdy dokument w bazie MongoDB musi mieć własność `_id`, stanowiącą jego unikalny identyfikator. Do własności tej może być przypisana wartość **dowolnego typu**; domyślnie jest to 12-bajtowe tzw. `ObjectId`.

Jeśli własność `_id` nie jest jawnie podana przy tworzeniu dokumentu, to zostaje ona dodana automatycznie.

Oprogramowanie

Na stronie internetowej:

mongodb.com

można znaleźć instrukcje instalacji oprogramowania związanego z systemem MongoDB. W skład tego oprogramowania wchodzą m.in.:

- **MongoDB Community / Enterprise Server** (`mongod`) – „serwer" MongoDB, czyli program, który musi być uruchomiony, aby dało się korzystać z systemu MongoDB;
- **MongoDB Shell** (`mongosh`) – program obsługiwany w wierszu poleceń, umożliwiający wykonywanie operacji takich jak tworzenie baz danych, wypełnianie ich dokumentami itd.;
- **MongoDB Compass** – aplikacja z graficznym interfejsem użytkownika, służąca do odczytywania, modyfikacji i analizy zawartości baz danych.

Ponadto, aby móc obsługiwać system MongoDB za pomocą tworzonej przez siebie aplikacji, należy ściągnąć sterownik (ang. *driver*) przeznaczony dla wybranego języka programowania lub środowiska. Obsługa MongoDB za pomocą Node.js jest omówiona w następnym podrozdziale.

Serwer MongoDB działa domyślnie na porcie **27017**.

MongoDB Shell

Po uruchomieniu w wierszu poleceń programu `mongosh` można wykonywać np. następujące operacje:

- połączenie z wybraną bazą danych;

```
> use [nazwa_bazy_danych];
> use music;
```

- zapis nowego dokumentu;

```
> db.[nazwa_kolekcji].insertOne([dane w formacie JSON]);
> db.bands.insertOne({name: "Led Zeppelin", formed: 1968});
```

- wyświetlenie wszystkich dokumentów w kolekcji;

```
> db.[nazwa_kolekcji].find();
```

- wyświetlenie pojedynczego dokumentu o zadanych własnościach;

```
> db.[nazwa_kolekcji].findOne({[poszukiwane własności]});
> db.bands.findOne({formed: 1968});
```

- usunięcie dokumentu o zadanych własnościach;

```
> db.[nazwa_kolekcji].deleteOne({[zadane własności]});
> db.bands.deleteOne({_id: ObjectId('66d85657cba0ae98f82710bd')});
```

- zmiana zawartości dokumentu;

```
> db.[nazwa_kolekcji].updateOne({[własności modyfikowanego dokumentu]}, {$set: {[nowe własności]}});
> db.bands.updateOne({name: "Led Zeppelin"}, {$set: {vocal: "Robert Plant"}});
```

- wyświetlenie listy baz danych;

```
> show dbs;
```

- wyświetlenie listy kolekcji;

```
> show collections;
```

- wczytanie i wykonanie instrukcji, zawartych w pliku .JS (w języku JavaScript).

```
> load([ścieżka pliku z kodem JavaScript]);
> load("scripts/prepare_db.js");
```

Szczegółowe informacje na temat obsługi programu MongoDB Shell można znaleźć w dokumentacji, dostępnej pod adresem:

mongodb.com/docs/mongodb-shell/

W systemie MongoDB (w przeciwieństwie np. do systemu MySQL) nie trzeba jawnie tworzyć bazy danych ani kolekcji – MongoDB zrobi to automatycznie przy ich pierwszym użyciu. Przykładowo, jeśli w momencie wywołania polecenia:

```
> use music
```

nie ma jeszcze bazy danych o nazwie *music*, to zostanie ona automatycznie utworzona.

Typy danych

W dokumentach MongoDB mogą być przechowywane dane tych samych typów, co w formacie JSON, czyli:

- dane tekstowe (ang. *string*),
- liczby,
- wartości binarne (`true` / `false`),
- wartość `null`,
- tablice,
- całe inne dokumenty,

a także inne, m.in. daty (`new Date()`).

Typu danych nie trzeba jawnie deklarować przy ich zapisie.

Dokumentacja MongoDB:
mongodb.com/docs

Obsługa MongoDB za pomocą Node.js

Sterownik

Do obsługi systemu MongoDB za pomocą środowiska Node.js służy pakiet `mongodb` (nazywany sterownikiem, ang. *driver*), który można zainstalować za pomocą npm:

```
> npm install mongodb
```

W kodzie JavaScript należy zaimportować klasę `MongoClient` z tego pakietu:

```
const { MongoClient } = require("mongodb");
```

Nawiązanie połączenia

Aby nawiązać połączenie, należy sformułować tzw. identyfikator połączenia (ang. *connection URI* albo *connection string*), czyli zmienną tekstową, zawierającą informacje o adresie serwera, numerze portu i innych parametrach połączenia – na przykład:

```
const uri = "mongodb://localhost:27017";
```

...a następnie za jej pomocą utworzyć obiekt klasy `MongoClient`:

```
const client = new MongoClient(uri);
```

Asynchroniczne wykorzystanie połączenia

Pakiet `mongodb` wykorzystuje *promises* do realizacji asynchronicznej komunikacji z bazą danych. Dzięki temu, można tworzyć czytelny kod programu z użyciem słów `async`, `await`, `try`, `catch` itp. Szkielet fragmentu programu, realizującego komunikację z bazą MongoDB, może wyglądać następująco (p. dokumentacja):

```
async function run() {
  try {

    // ... tu należy realizować komunikację z bazą danych
    // za pomocą obiektu client, używając "await"

  } finally {
    await client.close();  // zakończenie połączenia
  }
}

run().catch(console.log)  // wypisanie ewentualnych błędów
```

Komunikacja z bazą danych

Do komunikacji z bazą MongoDB można wykorzystać obiekt `MongoClient` podobnie, jak obsługuje się taką bazę za pomocą programu MongoDB Shell (omówionego w poprzednim podrozdziale). Na przykład, poniższy fragment kodu służy do tego, aby wyszukać w kolekcji *bands* bazy danych *music* dokument, w którym własność *name* ma wartość *Queen*:

```
var bands = client.db("music").collection("bands");
var data = await bands.findOne({name: "Queen"});
console.log(data)
```

Listę podstawowych metod komunikacji z bazą danych MongoDB można znaleźć w dokumentacji:

Node.js Driver – Quick Reference

Pełny przykład

Poniższy przykład stanowi zebranie fragmentów kodu, omówionych wyżej.

```
const { MongoClient } = require("mongodb");

const uri = "mongodb://localhost:27017";
const client = new MongoClient(uri);

async function run() {
  try {
    var bands = client.db("music").collection("bands");
    var data = await bands.findOne({name: "Queen"});
    console.log(data)

  } finally {
    await client.close();
  }
}

run().catch(console.log)
```

Więcej informacji:

- Dokumentacja MongoDB – Node Driver
- Dokumentacja MongoDB – Using Mongo with Node.js

Inne techniki back-endowe

Język JavaScript i środowisko Node.js to tylko jedne z licznych narzędzi, które można wykorzystać do realizacji back-endu. Poniżej wymienione jest kilka popularnych alternatyw.

Python

Python jest językiem programowania powszechnie lubianym m.in. ze względu na łatwość opanowania go przez początkujących programistów. Postaw języka Python można się nauczyć np. korzystając z oficjalnego samouczka: **The Python Tutorial**.

Do tworzenia aplikacji internetowych w języku Python można wykorzystać **Django**.

Java

Java językiem programowania szeroko stosowanym od dawna (jak na języki programowania). Istnieją zarówno liczne materiały internetowe, jak i dobre podręczniki, poświęcone nauce programowania w Javie.

Do tworzenia aplikacji internetowych w języku Java można wykorzystać **Spring**.

C#

C# jest językiem programowania opracowanym przez pracowników firmy Microsoft.

Do tworzenia aplikacji internetowych w języku C# można wykorzystać **ASP.NET**.

PHP

PHP (skrót od *PHP Hypertext Processor*) jest językiem programowania nadającym się przede wszystkim do tworzenia stron internetowych i aplikacji sieciowych: skrypty w języku PHP można m.in. przeplatać z kodem HTML.

Do tworzenia aplikacji internetowych w języku PHP można wykorzystać **Laravel**.

Ruby

Do tworzenia aplikacji internetowych w języku Ruby można wykorzystać **Ruby on Rails**.